

HyCAN: A Hybrid Approach for CAN Bus Intrusion Detection Using performance analysis of Machine Learning Techniques

By

Vikrant

A Thesis

Submitted to the Faculty of Graduate Studies
through the School of Computer Science
in Partial Fulfillment of the Requirements for
the Degree of Master of Science in Computer Science
at the University of Windsor

Windsor, Ontario, Canada

2024

©2024 Vikrant

HyCAN: A Hybrid Approach for CAN Bus Intrusion Detection Using performance
analysis of Machine Learning Techniques

by

Vikrant

APPROVED BY:

M Hassanzadeh
Electrical and Computer Engineering

M Assaduzzaman
School of Computer Science

I Saini, Advisor
School of Computer Science

October 13, 2024

DECLARATION OF CO-AUTHORSHIP

I. Co-Authorship

I hereby declare that this thesis incorporates material that is the result of research conducted under the supervision of Dr. Ikjot Saini. In all cases, the author performed the key ideas, primary contribution, experimental designs, data analysis, and interpretation, and the co-author's contribution was primarily through providing feedback and proofreading of the published manuscripts.

I am aware of the University of Windsor Senate Policy on Authorship, and I certify that I have properly acknowledged the contribution of other researchers to my thesis and have obtained written permission from each of the co-author(s) to include the above material(s) in my thesis. With the above qualification, I certify that this thesis and the research to which it refers is the product of my work.

II. Previous Publication

I certify that I have obtained written permission from the copyright owner(s) to include the above-published material(s) in my thesis. I certify that the above material describes work completed during my registration as a graduate student at the University of Windsor.

III. General

I hereby certify that I am the sole author of this thesis and that no part of this thesis has been published or submitted for publication. I certify that, to the best of my knowledge, my thesis does not infringe upon anyone's copyright nor violate any proprietary rights and that any ideas, techniques, quotations, or any other material from the work of other people included in my thesis, published or otherwise, are fully acknowledged in accordance with the standard referencing practices. Furthermore, to the extent that I have included copyrighted material that surpasses the bounds of fair dealing within the meaning of the Canada Copyright Act, I certify that I have obtained written permission from the copyright owner(s) to include such material(s) in my thesis and have included copies of such copyright clearances to my appendix.

I declare that this is a true copy of my thesis, including any final revisions, as approved by my thesis committee and the Graduate Studies office, and that this thesis has not been submitted for a higher degree to any other University or Institution. I understand that my thesis may be made electronically available to the public.

ABSTRACT

Electronic Control Units (ECUs) have become important to modern vehicles, enhancing operational control, driving comfort, and safety. These ECUs communicate using the Controller Area Network (CAN) protocol, which, despite its widespread adoption, is vulnerable to various security threats. According to the Upstream 2023 report, the number of high and massive-scale incidents that potentially impacted thousands of mobility assets increased by x2.5 compared to 2022. While developing a more secure CAN protocol is crucial, intrusion detection systems (IDS) offer a viable path to mitigating cyberattacks on vehicles. This thesis proposes a Hybrid Framework for machine learning-based IDS employing a range of models, including K-Nearest Neighbor (KNN), Logistic Regression, Decision Tree (DT), Random Forest, XGBoost, CatBoost, Multi-Layer Perceptron (MLP), Long Short-Term Memory (LSTM), and Transformer-based approaches. Our research used the newly published can-train-test dataset specifically designed for building ML-based IDS. The dataset consists of nine different types of attacks performed on four different vehicles in a real-world environment. This study majorly focused on DoS, Fuzzy, Gear Spoofing, Speed Spoofing and Standstill attacks performed on Chevrolet Impala and Subaru Forester. Notably, proposed IDS, which leverages both traditional machine learning models and advanced deep learning architectures, achieves good accuracy with less training and prediction time, exhibiting a high true positive rate and a low false negative rate. Additionally, our novel approach involves converting CAN bus data into feature strings for classification using transformers-based models, providing enhanced interpretability and robustness. The proposed Hybrid Framework is evaluated using various performance metrics, including accuracy, precision, recall, F1-score, and the area under the receiver operating characteristic curve (AUC-ROC). The results demonstrate that the proposed Hybrid Framework offers a more effective and efficient solution for detecting cyberattacks on vehicles.

DEDICATION

I would like to dedicate this thesis to my family. This achievement would not have been possible without their constant love and support. They have always believed in me and encouraged me to follow my dreams, no matter how impossible they may have seemed. Their unwavering faith and encouragement sustained me through the countless late nights and setbacks, and their sacrifices allowed me to pursue my education. I am forever grateful for all they have done for me and could not have reached this moment without them.

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to the following individuals who have provided me with unwavering support throughout my Master's thesis journey.

First and foremost, I would like to thank my supervisor Dr.Ikjot Saini, for her guidance throughout my research work. She gave me the freedom to choose the subject of my research, and her ongoing support and direction encouraged me to refine my work. She had a deep grasp of research challenges, which taught me a lot, and I appreciate her help and support throughout. I am grateful for the knowledge she shared with me and the time she invested in me.

I would also like to extend my heartfelt thanks to my committee members, Dr. Mohammed Hassanzadeh and Dr.Muhammad Assaduzzaman, for their valuable time and support. Their feedback and suggestions have been invaluable in shaping my research work.

I will always be grateful to my parents for their support, guidance, and understanding as well as for always being there for me. I also like to thank my younger brother and other family members for helping and constantly motivating me to reach my objectives.

Finally, I would like to thank my friends for their continuous support and encouragement as well as for bearing with me during my moments of frustration. It can be extremely challenging to live away from home, but with the support of my friends, it became a lot easier for me. Words cannot express how grateful I am to my parents, siblings, and friends for always supporting me in every situation. This thesis would not have been possible without their support and encouragement.

TABLE OF CONTENTS

DECLARATION OF CO-AUTHORSHIP	III
ABSTRACT	V
DEDICATION	VI
ACKNOWLEDGEMENTS	VII
LIST OF TABLES	XI
LIST OF FIGURES	XIII
LIST OF ABBREVIATIONS	XIV
1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	3
1.3 Relevance to Society and Research: Motivation	4
1.4 Thesis Contributions	5
1.5 Thesis Organization	6
2 Background Review	7
2.1 Overview of Vehicle Communication	7
2.2 In-vehicle Networks (IVNs)	7
2.2.1 Electronic Control Units (ECUs)	9
2.2.2 Controller Area Network (CAN) Bus	9
2.2.3 Vulnerabilities in CAN bus	11
2.2.3.1 Confidentiality	13
2.2.3.2 Authenticity and Non-repudiation	13
2.2.3.3 Integrity	13
2.2.3.4 Availability	13
2.2.4 Possible Intrusions in CAN bus	14
2.2.4.1 Denial of Service (DoS)	14
2.2.4.2 Message Injection attack	14
2.2.4.3 Replay attack	15
2.2.4.4 Sniffing attack	15
2.2.4.5 Spoofing attack	15
2.2.4.6 Fuzzing attack	16
2.2.4.7 Eavesdropping	16
2.2.4.8 Bus-off attack	16
2.2.4.9 Other attacks	17
2.3 Overview of ML and DL Models	17
2.3.1 Machine Learning Models	17

2.3.1.1	Logistic Regression (LR)	17
2.3.1.2	K-Nearest Neighbours (KNN)	19
2.3.1.3	Decision Trees (DT)	21
2.3.1.4	Random Forest (RF)	23
2.3.1.5	XGBoost	25
2.3.1.6	CatBoost	26
2.3.1.7	Stacking Methods	28
2.3.2	Deep Learning Models	29
2.3.2.1	Multi-Layer Perceptrons (MLP)	30
2.3.2.2	Long Short-Term Memory (LSTM)	31
2.3.3	Transformer-based Models	34
2.3.3.1	Bidirectional Encoder Representations from Transformers (BERT)	34
2.4	Literature Review	34
3	Methodology	43
3.1	Tools and Libraries	43
3.2	System Configuration	44
3.3	Dataset	44
3.4	Research Scenarios	46
3.4.0.1	RS1 - With Timestamp	46
3.4.0.2	RS2 - Without Timestamp	47
3.5	Proposed Methodology	47
3.5.1	Data Preprocessing	47
3.5.1.1	Feature Selection	49
3.5.1.2	Data Balancing	49
3.5.1.3	Data Normalization	49
3.5.1.4	Feature String Formation	49
3.5.1.5	Data Splitting	50
3.5.2	Model Construction	50
3.5.2.1	Machine Learning Models	51
3.5.2.2	Deep Learning Models	53
3.5.2.3	Transformer-based Models	56
3.5.3	Model Evalaution	59
4	Experiments and Results	63
4.1	Results for Impala	63
4.1.1	DoS Attack	64
4.1.2	Fuzzy Attack	64
4.1.3	Gear Spoofing Attack	66
4.1.4	Speed Spoofing Attack	70
4.1.5	Standstill Attack	73
4.2	Results for Forester	73
4.2.1	DoS Attack	76
4.2.2	Fuzzy Attack	76

4.2.3	Gear Spoofing Attack	78
4.2.4	Speed Spoofing Attack	80
4.2.5	Standstill Attack	82
4.3	Results Discussion	85
4.4	Difference from other existing approaches	87
5	Conclusion and Future Work	89
5.1	Conclusion	89
5.2	Future Work	91
	REFERENCES	93
	VITA AUCTORIS	100

LIST OF TABLES

3.1	The Attributes of can-train-and-test	45
4.1	I.RS1.DoS - Impala results for Research Scenario 1 for DoS attack . .	65
4.2	I.RS2.DoS - Impala results for Research Scenario 2 for DoS attack . .	65
4.3	I.RS1.Fuzzy - Impala results for Research Scenario 1 for Fuzzy attack	67
4.4	I.RS2.Fuzzy - Impala results for Research Scenario 2 for Fuzzy attack	67
4.5	I.RS1.Gear - Impala results for Research Scenario 1 for Gear Spoofing attack	68
4.6	I.RS2.Gear - Impala results for Research Scenario 2 for Gear Spoofing attack	69
4.7	I.RS1.Speed - Impala results for Research Scenario 1 for Speed Spoofing attack	71
4.8	I.RS2.Speed - Impala results for Research Scenario 2 for Speed Spoofing attack	72
4.9	I.RS1.Standstill - Impala results for Research Scenario 1 for Standstill attack	74
4.10	I.RS2.Standstill - Impala results for Research Scenario 2 for Standstill attack	75
4.11	F.RS1.DoS - Forester results for Research Scenario 1 for DoS attack .	77
4.12	F.RS2.DoS - Forester results for Research Scenario 2 for DoS attack .	77
4.13	F.RS2.Fuzzy - Forester results for Research Scenario 1 for Fuzzy attack	79
4.14	F.RS2.Fuzzy - Forester results for Research Scenario 2 for Fuzzy attack	79
4.15	F.RS1.Gear - Forester results for Research Scenario 1 for Gear Spoofing attack	80
4.16	F.RS2.Gear - Forester results for Research Scenario 2 for Gear Spoofing attack	81
4.17	F.RS1.Speed - Forester results for Research Scenario 1 for Speed Spoof- ing attack	83

4.18	F.RS2.Speed - Forester results for Research Scenario 2 for Speed Spoof-	
	ing attack	84
4.19	F.RS1.Standstill - Forester results for Research Scenario 1 for Standstill	
	attack	85
4.20	F.RS2.Standstill - Forester results for Research Scenario 2 for Standstill	
	attack	86

LIST OF FIGURES

2.1	In-vehicle Network Architecture [1]	8
2.2	CAN Frame [24]	10
2.3	Representation of Logistic Regression Classification [29]	18
2.4	Representation of KNN Classification [35]	20
2.5	Representation of Decision Trees Classification [38]	21
2.6	Representation of Random Forest Classification [35]	24
2.7	Representation of XGBoost Classifier [40]	24
2.8	Representation of CatBoost Classifier [43]	26
2.9	Representation of Stacking Method [47]	28
2.10	Representation of MLP Classifier [50]	30
2.11	Representation of LSTM Memory Cell [51]	32
2.12	Transformer Architecture [52]	33
3.1	Research Scenarios	46
3.2	Proposed Framework	48
3.3	Illustration of BERT Tokenization on random numerical feature string	50
3.4	Tested Hyperparameters	51
3.5	Code snippet of BERT Tokenizer	58
3.6	Code snippet of BERT Model Loading	58
3.7	Representation of results for each attack type for both vehicles. . . .	62

LIST OF ABBREVIATIONS

IDS	Intrusion Detection System
ECU	Electronic Control Unit
CAN	Controller Area Network
ML	Machine Learning
DL	Deep Learning
NLP	Natural Language Processing
KNN	K- Nearest Neighbour
LR	Logistic Regression
DT	Decision Tree
RF	Random Forest
XGBOOST	Extreme Gradient Boosting
CatBoost	Categorical Boosting
MLP	Multi-Layer Perceptron
LSTM	Long Short-Term Memory
BERT	Bidirectional Encoder Representations from Transformers
RS	Research Scenario

CHAPTER 1

Introduction

1.1 Introduction

Modern vehicles have seen remarkable advancements in automotive technology, resulting in more perceptive cars equipped with a broader range of practical, cutting-edge applications [1]. These vehicles use hundreds of ECUs to control various features, all interconnected through a CAN bus. ECUs enhance a vehicle's subsystem by improving energy efficiency and reducing vibration and noise [2].

While the increased use of ECUs has significantly improved vehicle functionalities, it has also made vehicles more vulnerable to cyber-attacks [3]. The CAN protocol lacks security measures like encryption and authentication, exposing it to various cyber threats. Researchers have demonstrated serious security vulnerabilities in in-vehicle networks [4]. For instance, an attacker can inject fake messages into the CAN bus or manipulate and read an ECU through vulnerable interfaces like CD players, USB ports, and OBD-II port.

With the rise of wireless technologies such as Bluetooth, Wi-Fi, and 5G, vehicles have become even more advanced and can interact with their surroundings. However, this also increases their susceptibility to attacks. For example, vehicle key fobs have been used to hack into live systems [5]. ECUs can receive any broadcasting signals on the same bus, making malicious attacks like packet injection and data manipulation possible to confuse essential safety parts. Other attack methods include Radio, Tire Pressure Monitoring Systems (TPMS), GPS, and hacking of electronic windows, steering, and brakes [6]. These attacks pose significant risks to both the vehicle and

the driver, highlighting the need for effective intrusion detection systems (IDS) to safeguard vehicles and human lives.

Vehicles are vulnerable to numerous types of attacks, such as Denial-of-Service (DoS) attacks, flooding attacks, fuzzy attacks, spoofing attacks, and malfunction attacks [4]. Several studies have explored the safety issues in inter- and intra-vehicular communications [7]. Intrusion detection sensors have gained attention for their effectiveness in detecting intrusions. Some approaches involve using advanced encryption standards [8] or lightweight block ciphers [9] to secure in-vehicle communication with limited resources. However, these methods may only be effective for specific threat models considered during the design stages [10].

Most existing works on CAN protocol security focus on physical factors like access controls and encryption [11], [12], [13], [14], [15], [16]. However, there is a need for more efficient and robust IDS. Physical access limitations and cryptography are not always effective with lightweight systems. Machine Learning (ML)-based IDS techniques address these limitations. These techniques can record the information's fundamental statistical properties to detect attacks. Various ML algorithms, such as SVM, DT, MLP, and RF, classify attack types [12]. Given the computational limitations of conventional ECUs, ML algorithms provide promising and effective solutions for vehicular networks.

Despite the availability of many ML approaches, a hybrid framework that contain multiple approaches to perform the performance comparative study to build robust IDS is still lacking. Improvements can be made by utilizing other ML models and large CAN dataset. This motivates us to investigate the use of multiple ML techniques, such as LR, RF, DT, KNN, MLP and BERT, with extensive CAN datasets to enhance security concerns with in-vehicle CAN buses. Additionally, most existing IDS models are developed for specific vehicle datasets, with little explanation of their efficacy when applied to other vehicle CAN data. To address this, our proposed work uses recently published CAN data [17], which contains data from multiple vehicles. This dataset is used to train and test our IDS models, which are then evaluated based on their performance metrics.

1.2 Problem Statement

As modern vehicles become more connected and reliant on ECUs, the threat of cyber-attacks on In-vehicle networks is growing rapidly. One of the most vulnerable parts of this system is the CAN bus, which is essential for communication between various components within a vehicle. Unfortunately, this crucial system is also a prime target for hackers, who can exploit its weaknesses to carry out attacks that might compromise vehicle safety.

While there's been a lot of research into using machine learning to detect these intrusions, most existing solutions have significant limitations. Many of these IDS focus on specific types of attacks, like DoS, but don't offer comprehensive protection against a broader range of threats, such as spoofing or message injection. Moreover, these systems are often tested in controlled environments that don't capture the complexities of real-world scenarios, raising questions about their effectiveness when actually deployed in vehicles.

This research aims to tackle these challenges by developing a hybrid framework that can use multiple ML models to perform the comparative performance study and to detect various types of CAN bus intrusions using data from actual vehicles. The goal is to create a system that isn't just good at spotting one kind of attack but can handle multiple types of threats in real time. To achieve this, the research will involve training and testing different machine-learning models on real CAN data from different vehicles, all within the same scenario. This approach will help determine how well these models can classify different attacks, compare their performance, and gain a better understanding of the nature of these intrusions.

Ultimately, this research is about laying the groundwork for more reliable and extensible framework that can be used to compare multiple ML approaches using similar CAN data to build robust IDS for cars. The insights gained could lead to the development of more secure In-vehicle communication networks, helping to make our cars safer as they become more connected and complex.

1.3 Relevance to Society and Research: Motivation

As vehicles become more advanced, they rely heavily on electronic systems to handle everything from engine performance to safety features. These systems are connected through the CAN bus, which is like the vehicle’s nervous system, ensuring everything communicates smoothly. But as our cars get smarter, they also become more vulnerable to cyberattacks, and that’s where the danger lies.

Imagine a scenario where an attacker gains control over the engine or brake systems while the vehicle is in motion. The potential outcomes are very frightening and may lead to an road accident. Attacks on the CAN bus, such as engine control hijacking, brake system compromise, or transmission control interference [4], can put not just the vehicle’s passengers but also other road users at risk. These scenarios highlight the urgent need for robust and reliable IDS that can operate in real-time and protect vehicles from such life-threatening intrusions.

Despite the criticality of this problem, existing research has significant gaps. Most current studies lack the ability to test and compare multiple machine learning models in a real-world context, making it challenging to identify the most effective solutions. Furthermore, many available datasets are synthetic, lack realism, and need to capture the variability found in actual vehicles, leading to IDS solutions that may not perform well in practice.

This research is motivated by the need to fill these gaps by developing a hybrid framework that can use multiple ML models to perform the comparative performance study and to detect various types of CAN bus intrusions using data from actual vehicles. By providing a comprehensive analysis of different machine learning models under the different research scenarios, this study aims to contribute to the development of more secure and resilient In-vehicle networks, ultimately enhancing the safety of modern vehicles.

1.4 Thesis Contributions

The objective of this research is to develop a Hybrid Framework for ML-based IDS to secure the In-vehicular network, such as the CAN network. This study evaluates the efficiency of intrusion detection using CAN dataset collected from real-time vehicles. The proposed approach trains on CAN dataset to identify the abnormal behaviour in CAN packets and then perform classification. The key contributions of this work are as follows:

1. A critical review of existing vehicular IDS to identify research gaps and develop a Hybrid Framework to build robust IDS using ML.
2. The first kind of study that assess multiple ML models in a hybrid framework on the dataset collected from real vehicles (a Subaru Forester car and a Chevrolet Impala car) running on the road.
3. Development of an ML-based Hybrid framework using multiple classifiers: LR, KNN, DT, RF, XGBOOST, CATBOOST, MLP, LSTM and BERT.
4. The first kind of study that addresses computational overhead limitations of models by incorporating a hybrid framework that selects and fine-tunes models to maintain a high level of performance while minimizing computational demands.
5. The first study to use Stacked Ensemble methods and to fine-tune Transformers-based models.
6. Detection of various attacks: DoS, Fuzzy, Gear Spoofing, Speed Spoofing, Standstill, and attack-free states.
7. Observing the significance of timestamp feature by introducing new research scenarios.

1.5 Thesis Organization

The remaining outline of this thesis is as follows. The 2nd chapter explains the essential and basic ideas of In-Vehicle Networks, vulnerabilities and possible intrusions in CAN bus, a survey of relevant work and the overview of different Machine Learning and Deep learning algorithms implemented in this study. Chapter 3 provides an description of the proposed methodology, implementation specifics and the evaluation methods of the thesis work , followed by Chapter 4, which includes discussion of the results. Finally, chapter 5 provides a conclusion and a prospective future study of the proposed methodology.

CHAPTER 2

Background Review

2.1 Overview of Vehicle Communication

An In-vehicle networks is a system where different communication technologies, sensors, and computing resources come together to create an innovative, cooperative environment within vehicles. These systems enable seamless communication between different components, ensuring they work smoothly to perform essential tasks.

Communication within an IVNs happens on multiple levels. For example, various parts of a vehicle, such as sensors, control units, and devices, must communicate in real-time to ensure the vehicle operates correctly. This type of internal communication is crucial for things like engine control, braking, and safety systems, and it can either be one-way or two-way, depending on the need. This communication between components within the vehicle is known as internal vehicular communication [18].

In addition to internal communication, vehicles today often connect to external systems, which serve purposes like diagnostics, firmware updates, or providing internet access for passengers. Modern vehicles increasingly feature internet connectivity, allowing passengers to stay connected while on the move. This connection with systems outside the vehicle is called external vehicular communication [18].

2.2 In-vehicle Networks (IVNs)

In-vehicle networks are complex communication systems that connect various ECUs inside a vehicle, allowing them to exchange information and control different func-

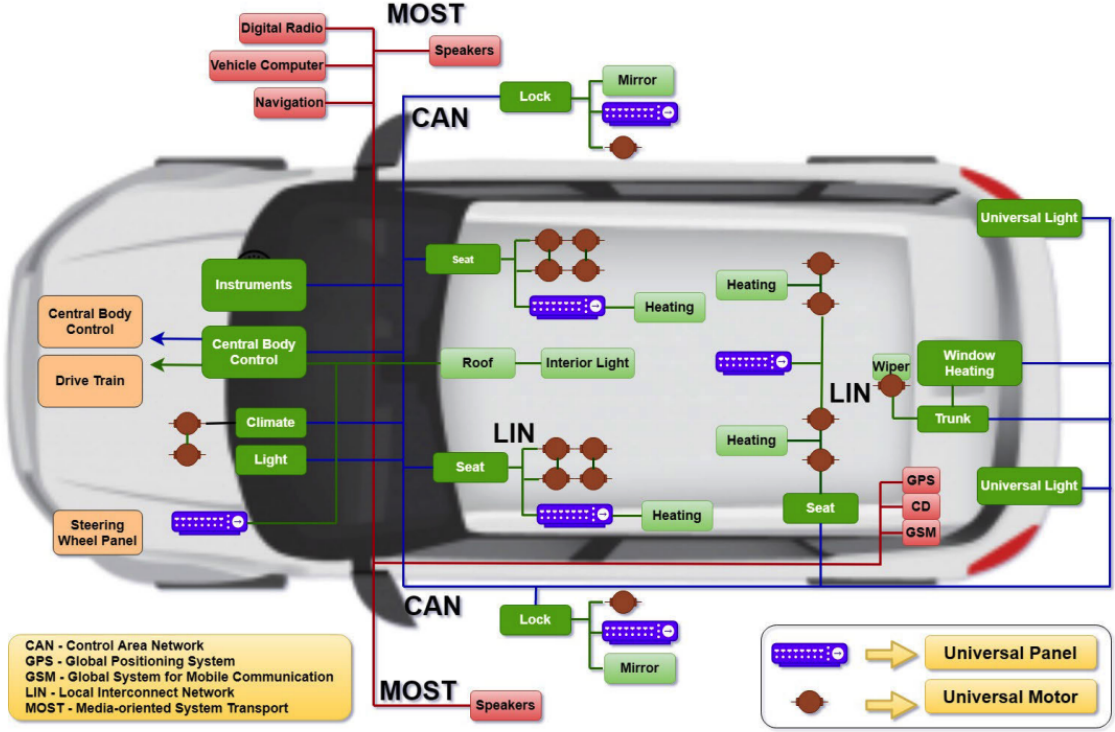


Figure 2.1: In-vehicle Network Architecture [1]

tionalities such as engine performance, braking, lighting, and air conditioning [19]. The architecture of an in-vehicle network is designed to ensure that these ECUs, sensors, and actuators can communicate effectively to perform their respective tasks in real time. The most commonly used network architectures in modern vehicles are based on protocols such as the Controller Area Network (CAN), Local Interconnect Network (LIN), FlexRay, Media-Oriented Systems Transport (MOST), and Ethernet [12]. Each of these protocols is chosen based on the specific communication requirements of the subsystems within the vehicle, including speed, reliability, and data bandwidth.

Modern vehicles integrate multiple network protocols (see Fig. 2.1), each catering to specific needs. Typically, the network architecture is hierarchical, with high-speed networks like CAN or FlexRay handling critical control systems (powertrain, safety systems) and lower-speed networks like LIN managing non-critical subsystems (lighting, HVAC).

2.2.1 Electronic Control Units (ECUs)

An Electronic Control Unit is a compact embedded computing system designed for real-time computing. It is distinguished by its ability to operate within strict time constraints and maintain low power consumption, making it integral to managing various automotive functions efficiently [19]. In modern vehicles, an ECU controls specific functionalities such as braking, airbag deployment, engine control, parking, and the door lock/unlock system [20]. Typically, a vehicle can contain up to 100 ECUs, which are used to manage sensors and actuators [21].

ECUs communicate with each other via the CAN bus. Each ECU comprises a CAN controller, a CAN transceiver, and a microcontroller. However, several researchers have demonstrated that it is possible to send fake messages to different ECUs using the In-vehicle network, read ECU memory, and access ECU security keys [20]. Therefore, attacks on the CAN bus can affect the vehicle's system and pose significant risks to driver safety.

2.2.2 Controller Area Network (CAN) Bus

The Controller Area Network bus is a serial communication protocol for real-time, safety-critical functions in road vehicles and other controlled applications [22]. It is a multi-master protocol, making it the most widely used vehicle communication system [20]. Moreover, CAN ensures reliable transmission and quick recovery with an efficient error detection mechanism, making it robust [23]. A microcontroller is connected to the CAN controller, which has two pins: the transmitter and the receiver. The CAN transceiver manages data communication to and from the bus, using differential voltages with two states: dominant (logical 0) and recessive (logical 1). These two pins on the CAN transceiver are directly connected to the bus, allowing the ECU to transmit and receive messages.

The CAN Bus protocol offers several key characteristics that make it suitable for various automotive applications:

- Bitrate and Payload: The classical CAN bus supports a maximum bitrate of 1

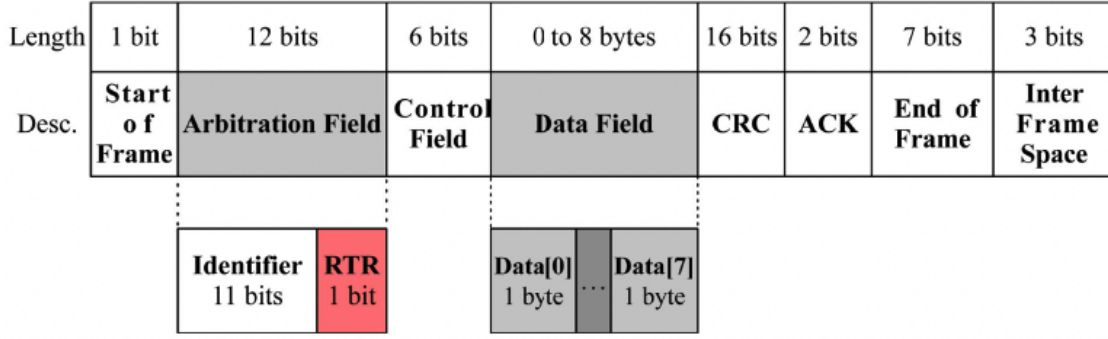


Figure 2.2: CAN Frame [24]

Mbps with a payload capacity of up to 8 bytes. High-speed CAN bus bitrates range from 125 kbps to 1 Mbps, while low-speed CAN bus bitrates range from 5 kbps to 125 kbps.

- **Sensor Connectivity:** Critical sensors are connected to the high-speed CAN bus to ensure they receive the necessary bandwidth and low latency, while less-critical sensors are connected to the low-speed CAN bus.
- **Access Mechanism:** The CAN bus protocol ensures real-time access for critical sensors through the Carrier Sense Multiple Access with Collision Avoidance and Arbitration Priority (CSMA/CA-AP) access/arbitration mechanism. This system allows multiple ECUs to communicate efficiently by prioritizing messages based on their CAN ID.
- **Physical Implementation:** The CAN bus is physically implemented using twisted pair cables, which helps reduce electromagnetic interference and ensures reliable communication.
- **Error Handling:** The CAN bus protocol includes robust error detection mechanisms, which ensuring data integrity and reliable communication even in the presence of errors.

CAN bus uses a message-based protocol [12] that utilizes the concept of priority filtering by assigning higher priority to specific IDs for message transmission. This

ensures reliable data transmission even when multiple nodes are simultaneously sending data. Along with the ID, a CAN frame comprises segments such as start of frame, data length code, data, cyclic redundancy check, acknowledgement, and end of frame. The specific structure of a CAN frame is illustrated in Fig. 2.2 and detailed below:

- The Start of Frame (SOF): A signaling bit marking the beginning of a data frame on the CAN bus.
- Arbitration field: Comprises 11 bits for identification and one Remote Transmission Request (RTR) bit.
- Identifier bits: These provide a unique ID to each message, and the RTR bit activates for a remote frame.
- Control field: Contains a 4-bit Data Length Code (DLC), which denotes the amount of data in the data field, and two reserved bits
- Data field: Houses the message's content, accommodating between 0 to 8 bytes.
- Cyclic Redundancy Check (CRC) field: Houses the CRC for error detection during transmission.
- Ack field (ACK): Includes an Acknowledgment bit, signifying the successful receipt of a message by receivers.
- End of Frame (EOF): A sequence of 7 bits marking the end of a message.

2.2.3 Vulnerabilities in CAN bus

The Controller Area Network bus, widely used in automotive and industrial systems for device-to-device communication, suffers from a number of major vulnerabilities due to a lack of native security features. The lack of built-in encryption and authentication introduces risks such as message spoofing, where an attacker could inject malicious messages that could potentially allow them to take unauthorized control of

vehicle functions such as braking or cornering. Additionally, the CAN bus is vulnerable to DoS attacks, where excessive message traffic can overload the system, delaying or blocking the transmission of critical control data. This vulnerability in error handling can lead to significant operational disruption, highlighting the need for enhanced security measures in CAN bus deployments.

In [12], the paper highlights several serious vulnerabilities in the CAN bus used in connected and automated vehicles (CAVs). These include CAN bus spoofing attacks, in which an attacker injects fraudulent messages due to a lack of authentication, thereby disrupting vehicle functionality; CAN bus tampering attacks, which involve sending random frames to disrupt the computer and affect vehicle operation; CAN bus frame spoofing attacks, in which false data is inserted into messages, thereby maliciously modifying vehicle parameters; CAN bus injection attacks, which exploit a lack of encryption to manipulate message transmissions, causing fraudulent events; and CAN bus DoS attacks, which overload the bus with high-priority messages, blocking legitimate communications and potentially disabling vehicle functionality. The paper emphasizes the importance of deploying robust IDS to detect and mitigate these attacks, as they are essential to maintaining the security and reliability of CAV.

The CAN bus protocol, widely used in automotive communications, is extremely vulnerable to a lack of message authentication and unencrypted communications, allowing attackers to inject spoofed messages and easily eavesdrop on traffic. This protocol and its design, which relies on broadcasting messages without a source or destination address, makes it vulnerable to replay attacks and unauthorized access. Additionally, the message arbitration method can be exploited to launch DoS attacks, in which an attacker repeatedly sends high-priority messages, thereby blocking legitimate communications. Vulnerabilities at the physical layer, such as jamming, further compromise the security of the CAN bus. These inherent weaknesses require additional security measures, such as IDS and cryptographic techniques, to protect against potential threats and keep vehicles safe [25].

Therefore, to conclude, the absence of security mechanisms leads to several potential vulnerabilities in CAN bus, which can be systematically examined in terms of

the Confidentiality, Integrity, and Availability [26] security model as:

2.2.3.1 Confidentiality

The CAN protocol inherently allows all nodes on the network to broadcast messages openly. Since messages are selected and processed based on their IDs, the system is susceptible to eavesdropping. This openness facilitates unauthorized parties to monitor and intercept communications effortlessly, posing a significant risk of data leakage.

2.2.3.2 Authenticity and Non-repudiation

The absence of mechanisms to verify sender identities introduces substantial risks to message integrity. Malicious actors can forge messages without traceability, sending fraudulent commands to any node within the network. This deficiency undermines the network's ability to assert the authenticity and origin of the messages, complicating non-repudiation.

2.2.3.3 Integrity

Attackers with access to network nodes, especially relay gateways, can manipulate message content. Although CAN employs Cyclic Redundancy Checks to detect errors, these checks do not prevent intentional tampering with the data payload. A compromised node can thus alter messages and still append a correct checksum, facilitating undetected data corruption.

2.2.3.4 Availability

The capability of attackers to send arbitrary data or exert control over gateway nodes can severely disrupt network operations. Such attacks might range from flooding the network to cause denial of service, to more sophisticated forms of interference where specific messages are blocked or altered, thus impacting the normal functionality of networked devices.

2.2.4 Possible Intrusions in CAN bus

The CAN bus is widely used in automotive and industrial systems for real-time communication between ECUs. However, the lack of built-in security features such as encryption, authentication, and message segmentation makes the CAN bus highly vulnerable to various forms of cyberattacks. These vulnerabilities allow malicious actors to exploit the system, potentially compromising the vehicle’s safety and control. Below, we discuss the most common and significant types of intrusions targeting the CAN bus, as highlighted in studies by [4] and [12].

2.2.4.1 Denial of Service (DoS)

In this attack, the attacker can exploit the arbitration process used by the CAN protocol, wherein messages are prioritized based on their ID [4]. The attacker sends high-priority CAN messages continuously, flooding the bus and preventing transmitting lower-priority legitimate messages. This results in service unavailability or system failure. For example, a DoS attack could block critical messages about engine control, brakes, or steering in a vehicle, leading to a potential safety hazard. Since CAN bus networks lack bandwidth throttling or queuing mechanisms, they are especially vulnerable to such attacks, making the prevention and detection of DoS attacks critical in vehicular security.

2.2.4.2 Message Injection attack

A Message Injection Attack involves injecting forged messages into the CAN bus. Since CAN lacks authentication mechanisms, attackers can send fabricated messages with valid CAN IDs, making them indistinguishable from legitimate traffic [27]. These malicious messages can target specific ECUs, potentially overriding critical control systems. For instance, an attacker could inject fake gear shift or speedometer messages, misleading the vehicle’s control systems. The simplicity of this attack, combined with the ability to manipulate real-time vehicle operations, underscores the need for robust intrusion detection mechanisms.

2.2.4.3 Replay attack

In a Replay Attack, an attacker captures legitimate CAN messages and replays them later to trick the system into executing unintended commands [27]. Since CAN messages do not contain timestamps to ensure freshness, replay attacks are challenging to detect. An attacker can capture a message related to, for example, door locking and replay it later to unlock the doors. Such attacks exploit the lack of message verification mechanisms, disrupting the intended vehicle behaviour.

2.2.4.4 Sniffing attack

This attack involves eavesdropping on CAN traffic. Due to the broadcast nature of the CAN protocol [27], all ECUs connected to the bus can hear messages from each other. Attackers can use this by passively monitoring the bus to collect sensitive information, such as the vehicle’s operational data or control messages. The unencrypted nature of CAN communication makes this attack particularly easy to execute. Sniffing attacks serve as a precursor to more severe attacks, as they allow adversaries to gain insights into the system’s functionality, which can be used to carry out more sophisticated intrusions, such as message injection or spoofing..

2.2.4.5 Spoofing attack

In this attack, an attacker impersonates a legitimate ECU by sending CAN messages with a valid identifier [27]. Since the CAN bus lacks message authentication, the vehicle’s control system cannot differentiate between genuine and fake messages. For example, an attacker could spoof messages from the brake ECU, causing the vehicle to misinterpret the system’s status. Spoofing attacks pose a severe threat, allowing attackers to manipulate vital vehicle functions, potentially leading to dangerous situations for drivers and passengers.

2.2.4.6 Fuzzing attack

A Fuzzing Attack involves sending random, malformed, or unexpected data to the CAN bus to test how the ECUs handle such inputs [27]. Fuzzing aims to discover vulnerabilities by analyzing the system’s response to invalid or corrupted messages. Fuzzing attacks can expose weaknesses in ECU firmware or software, leading to potential system crashes or unintended behaviour. In practice, attackers can use fuzzing to identify vulnerabilities in specific vehicle components, such as the airbag control unit, and exploit them to cause system malfunctions.

2.2.4.7 Eavesdropping

Eavesdropping occurs when an attacker passively monitors all CAN messages transmitted across the bus [4]. As the CAN protocol broadcasts messages to all connected ECUs, it allows an attacker to gather sensitive information about the vehicle’s operation, such as the driver’s inputs, vehicle speed, or system diagnostics. Since CAN messages are unencrypted, eavesdropping does not require any sophisticated techniques. This attack often serves as reconnaissance, enabling attackers to gather intelligence on the system before launching more intrusive attacks, such as message injection or spoofing.

2.2.4.8 Bus-off attack

A Bus-off Attack exploits the error-handling mechanisms of the CAN bus. When an ECU detects too many errors, the protocol forces it into a “bus-off” state, effectively disconnecting it from the network. In a bus-off attack, the attacker deliberately causes transmission errors by sending malformed messages or interfering with the bus’s electrical signals. During the attack, it can cause critical ECUs, such as those controlling the engine or brakes, to enter a bus-off state, rendering them inoperative. The consequences of such an attack can be dangerous, especially in scenarios where real-time control of the vehicle is essential for safety.

2.2.4.9 Other attacks

There can be other attacks like sensor manipulation [4], where the attackers targeting sensors connected to the CAN bus, such as Tyre Pressure Monitoring Systems (TPMS), can manipulate sensor data to send incorrect information to the vehicle's control systems [4].

2.3 Overview of ML and DL Models

In this section we will review the different Machine Learning and Deep Learning models used in this research work. We compared the ten distinct algorithms performances in this research and tested different sets of hyperparameters for each algorithm to find the best set. The algorithms we used are as follows:

2.3.1 Machine Learning Models

Machine learning, a subset of Artificial Intelligence [28], enhances the ability of machines to perform tasks more efficiently and accurately through statistical learning. This technology and its subset, deep learning, are extensively utilized across various sectors. Machine learning algorithms can make predictions by analyzing input data to discover patterns, detect or categorize data, and address complex, real-world problems. These capabilities have made machine learning an indispensable tool in these industries, revolutionizing how businesses operate and protect information. In Vehicular Security, machine learning algorithms can detect attacks and misbehaviour in the network.

In this study, we used Supervised Machine Learning algorithms discussed below:

2.3.1.1 Logistic Regression (LR)

LR, often mistaken for a regression model, fundamentally operates as a linear classification method. Fig. 2.3 represents the logistic function, a sigmoid curve, to predict the probability of possible outcomes for a single trial [29]. This function is defined

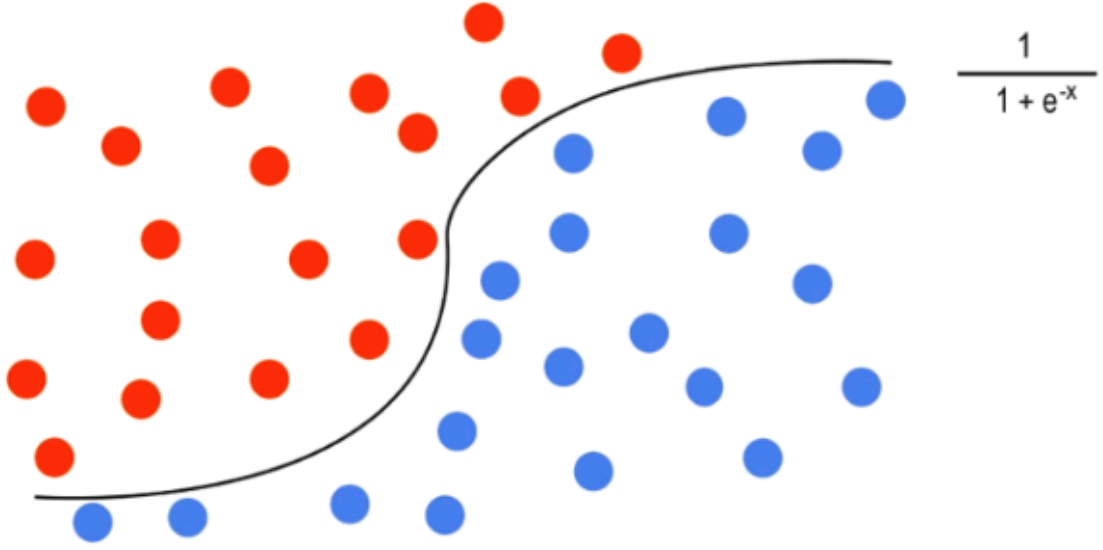


Figure 2.3: Representation of Logistic Regression Classification [29]

as:

$$f(x) = \frac{L}{1 + e^{-k(x-x_0)}}$$

where x_0 is the sigmoid's midpoint, L is the maximum curve value, and k indicates the logistic growth rate or the steepness of the curve.

Assuming $X_1, \dots, X_p$ as explanatory variables and Y as the binary response variable, logistic regression models the probability $P(Y = 1)$ given the predictors as:

$$P(Y = 1 | X_1, \dots, X_p) = \frac{\exp(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}{1 + \exp(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}$$

Here, $\beta_0, \beta_1, \dots, \beta_p$ are the regression coefficients estimated from the data using maximum likelihood estimation.

For new instances, classification into $Y = 1$ or $Y = 0$ is determined by comparing the modelled probability to a pre-set threshold c :

$$Y = 1 \text{ if } P(Y = 1) > c, \quad Y = 0 \text{ if } P(Y = 1) \leq c$$

The effectiveness of logistic regression depends on the model's fit to the data. Once fitted, the “predict_proba” method can predict the probability of a positive outcome [30]. A regularization term $r(w)$ is included to manage overfitting and is

integrated into the minimized cost function during training:

$$\min C \sum_{i=1}^n (-y_i \log(\hat{p}(X_i)) - (1 - y_i) \log(1 - \hat{p}(X_i))) + r(w)$$

Regularization controls the model's complexity, improving its generalization capabilities and reducing overfitting risks [31].

2.3.1.2 K-Nearest Neighbours (KNN)

KNN algorithm is a simple yet effective instance-based supervised machine learning algorithm used for multi-class classification problems. Fig. 2.4 represents KNN, which operates by calculating the distance between a new sample and existing samples in the training set. It assigns a class to the new sample on the basis of majority class samples in its neighbor. This method is an instance-based learning technique that makes no assumptions about the underlying distribution of the data [32].

Distance Metrics

KNN relies heavily on the distance metric used to identify the k nearest neighbors. The most commonly used distance metric is the Euclidean distance [33], defined as follows:

$$D_{\text{Euclidean}}(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

where p and q are the points being compared, each having n features.

Other distance metrics include:

- **Manhattan Distance:** This is calculated as the sum of the absolute differences between points across all dimensions [34].

$$D_{\text{Manhattan}}(x, y) = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$

- **Minkowski Distance:** A generalization of both Euclidean and Manhattan dis-

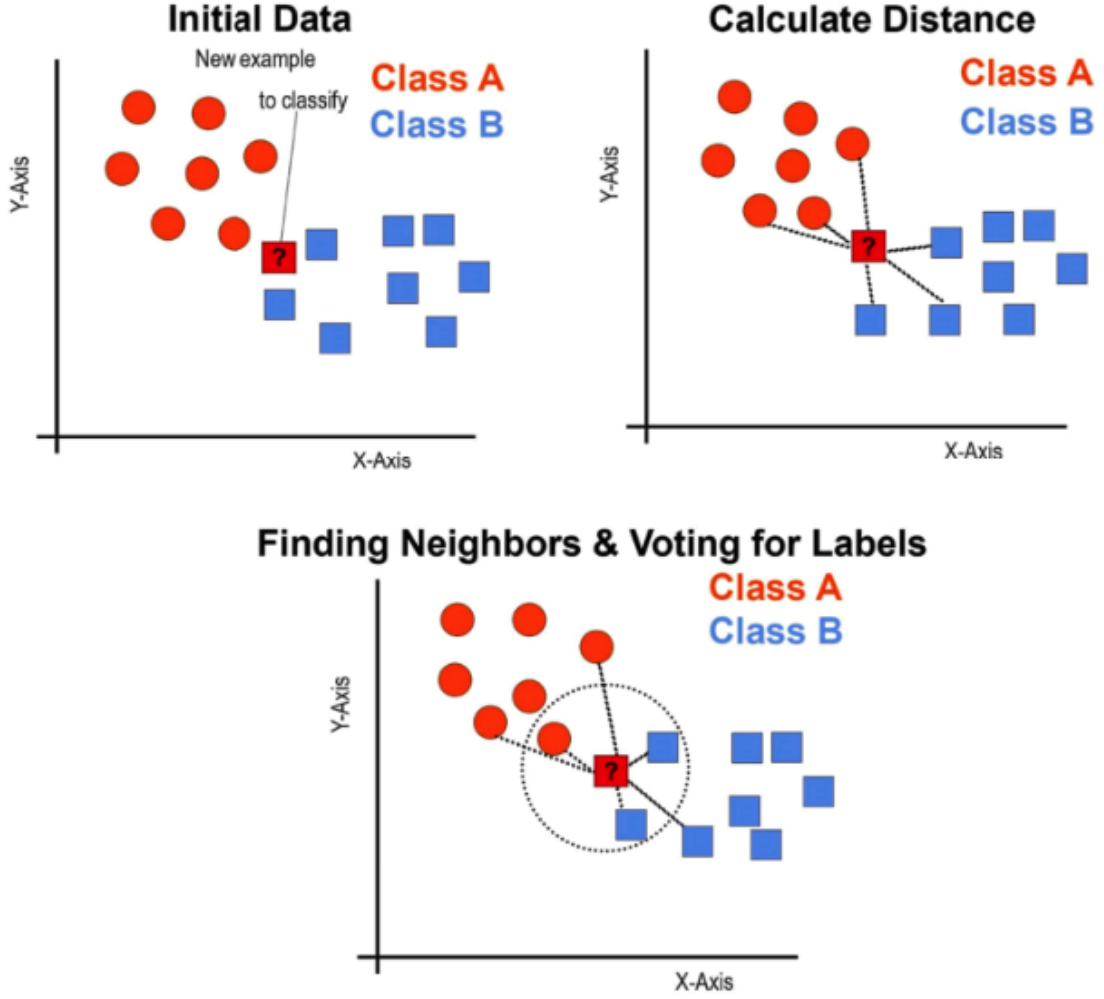


Figure 2.4: Representation of KNN Classification [35]

tances, defined for a parameter p as [34]:

$$D_{\text{Minkowski}}(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

- Jaccard Distance: Used for measuring the dissimilarity between sets, it is calculated as [34]:

$$D_{\text{Jaccard}}(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

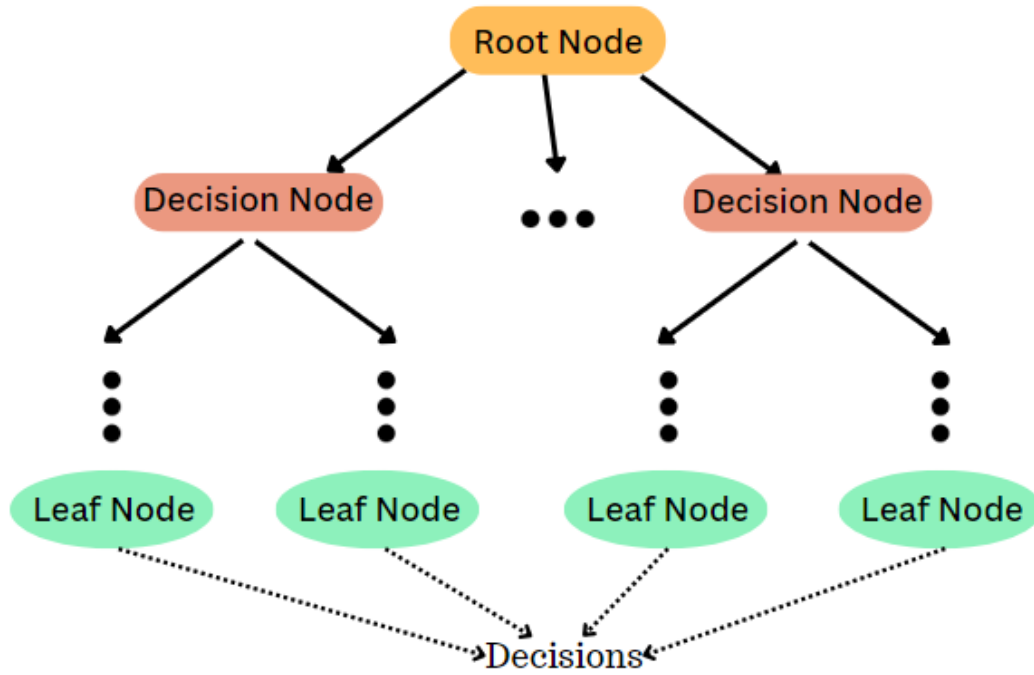


Figure 2.5: Representation of Decision Trees Classification [38]

Choosing the Right k

The choice of k is crucial in determining the effectiveness of the KNN algorithm. A larger k value can reduce the effect of noise on the classification, but it also makes the boundary between different classes less distinct. A common heuristic for selecting k is to set it to the square root of the number of features in the dataset, which balances between overfitting and underfitting [36].

KNN's simplicity and effectiveness make it widely used in many applications. However, its performance is highly dependent on the choice of distance function and the value of k .

2.3.1.3 Decision Trees (DT)

A decision tree is a supervised machine-learning algorithm that employs a tree-like structure to make decisions or predictions based on input features [37].

Fig. 2.5 shows a decision tree is essentially a set of if-then-else decision rules. The

topmost decision node in a tree that corresponds to the best predictor called root node. Decision trees handle both categorical and continuous data. The dataset is split up at each node according to a decision rule. This process is repeated on each derived subset in a recursive manner called recursive partitioning. The recursion process stops when any subset node has the same value as the target variable or when the stopping criterion is met.

The primary challenge in learning a decision tree is determining an optimal splitting strategy. This is generally achieved using a metric that measures the impurity of the nodes and the information gain from each split. Common metrics include:

- Gini Impurity: Used in classification, particularly in the CART (Classification and Regression Trees) algorithm. It measures how many times a randomly chosen element from the set would be identified incorrect.

$$G = 1 - \sum_{i=1}^n p_i^2$$

- Entropy: Another measure used in classification that quantifies the randomness or uncertainty in the data.

$$H = - \sum_{i=1}^n p_i \log_2 p_i$$

- Mean Squared Error: Used in regression tasks.

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

One of the main advantages of decision trees is their interpretability [38]. They are easy to understand and visualize, even for people without statistical knowledge, making them useful in decision support systems. However, trees can become very non-general if they are too deep, leading to overfitting. Techniques such as pruning *i.e.* removing parts of the tree that do not provide additional power or setting a maximum depth can help prevent this problem.

2.3.1.4 Random Forest (RF)

RF is a prominent machine learning method that utilizes ensemble learning techniques for classification and regression tasks. It aggregates multiple de-correlated decision trees, commonly called “bagging” to improve the model’s accuracy and robustness. RF is known for its simplicity in training and tuning compared to other methods such as boosting [35].

In Random Forest, each tree is built from a randomly sampled subset of the training data, using a random subset of features, to make sure that the trees are not correlated. This randomization helps reduce variance without substantially increasing bias. With the increase in number of trees, the generalization error of the forest converges to a limit, suggesting the stability of the model [39].

The prediction for an unobserved example x' is made by averaging the predictions from all the regression trees in the forest:

$$\hat{p} = \frac{1}{B} \sum_{b=1}^B f_{\sigma}(x')$$

where B is the number of trees and f_{σ} denotes the individual tree predictions.

Fig. 2.6 shows an RF classification of a new sample using majority voting. Random Forests can handle large data sets with excellent accuracy, even when data is missing, making it a preferred choice for practical machine learning challenges. The method also allows for the estimation of prediction uncertainty, calculated as the standard deviation of the predictions from all the trees on the unobserved example x' :

$$\sigma_{\mu} = \sqrt{\frac{\sum_{b=1}^B (f_{\sigma}(x') - \hat{f})^2}{B - 1}}$$

This measure of variability provides an indication of the confidence in the predictions made by the model.

Random Forests are not only effective for prediction but are also beneficial in feature selection due to their ability to naturally rank the importance of features based on how they improve the purity of the node, which in turn contributes to the

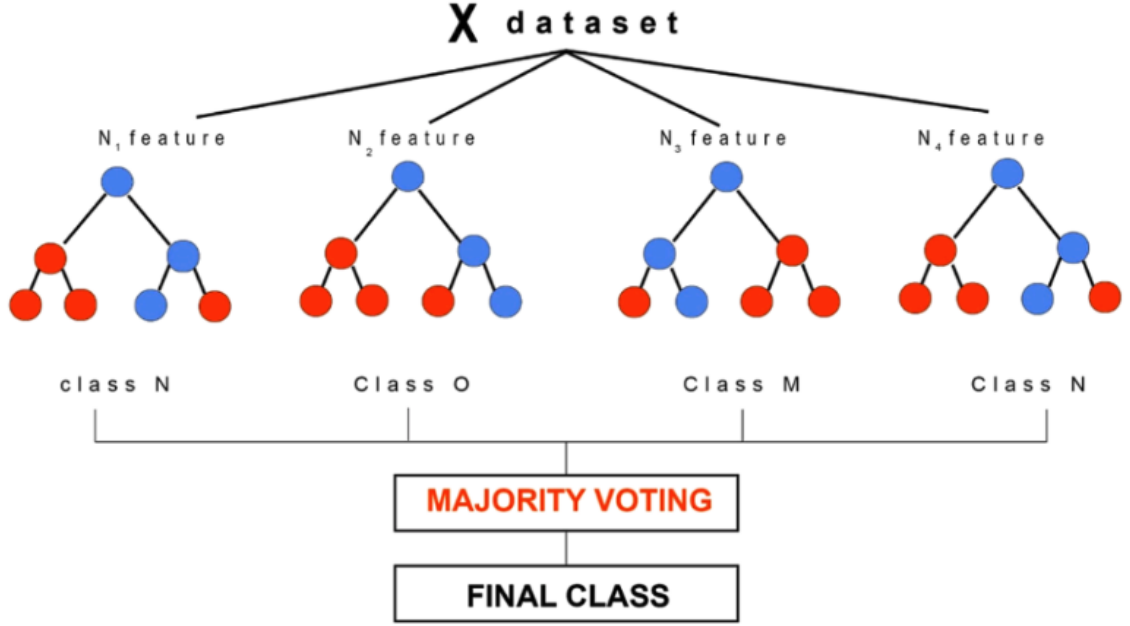


Figure 2.6: Representation of Random Forest Classification [35]

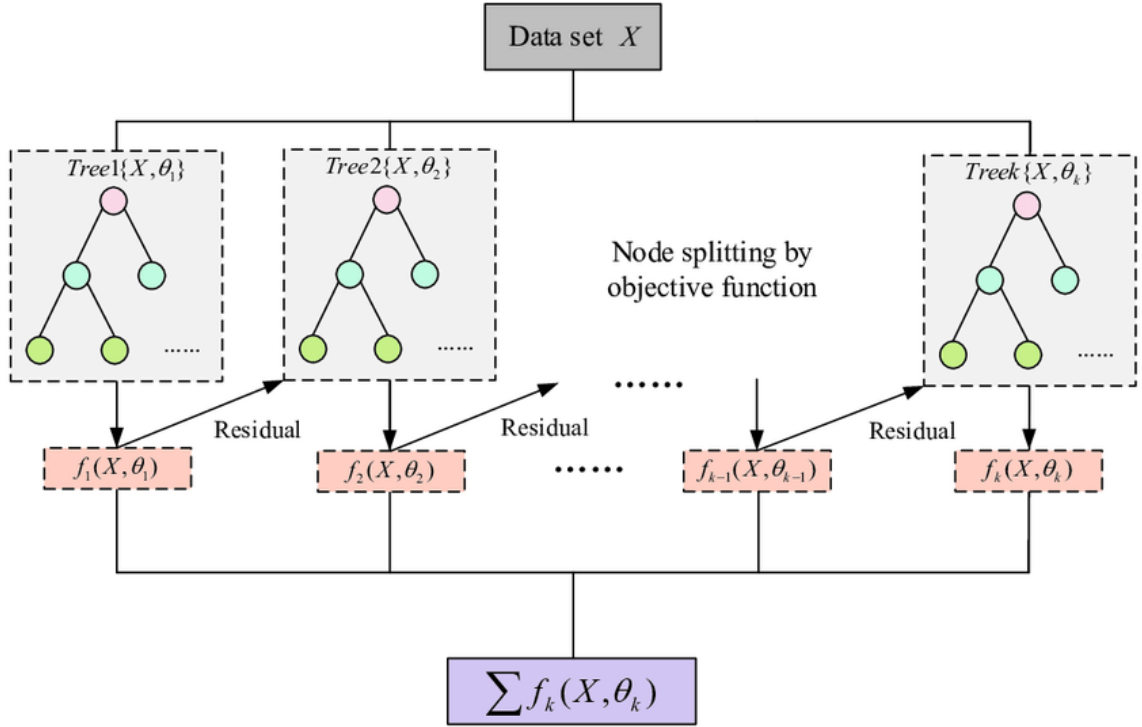


Figure 2.7: Representation of XGBoost Classifier [40]

accuracy of the model.

2.3.1.5 XGBoost

eXtreme Gradient Boosting, or XGBoost, is an efficient and scalable implementation of Friedman’s gradient boosting machines [41]. Fig. 2.7 represents an ensemble of decision trees sequentially [42], where each tree is built to correct the errors made by the previous trees. The method assigns weights to all independent variables, which are then used to build a decision tree that predicts outcomes. Incorrect predictions by a tree are given higher weights in subsequent trees, thus incrementally improving the model’s accuracy with each step [40].

The final prediction $\hat{y}_i(t)$ by XGBoost for a binary classification task is given as:

$$\hat{y}_i(t) = \sum_{k=1}^t f_k(x_i)$$

where $f_k(x_i)$ represents the contribution of the k -th tree.

In gradient boosting, the cumulative prediction function up to the m -th tree is denoted as:

$$F_M(x_i) = \sum_{i=1}^m h_m(x_i)$$

Although the intermediate trees output continuous values, the final classification is derived by mapping these continuous outputs to class probabilities using a loss-dependent function. Using the log-loss model, the probability that x_i belongs to the positive class is computed using the sigmoid function:

$$p(y_i = 1 \mid x_i) = \sigma_F(F_M(x_i))$$

where σ_F is the sigmoid function, adapting the continuous output $F_M(x_i)$ to the interval $[0, 1]$.

Regularization techniques enhance XGBoost’s performance by scaling the contribution of each tree. This is regulated using the ν parameter, which affects the step length in the gradient descent procedure, akin to the learning rate. This parameter is one of the most critical hyperparameters in XGBoost and is typically set using the

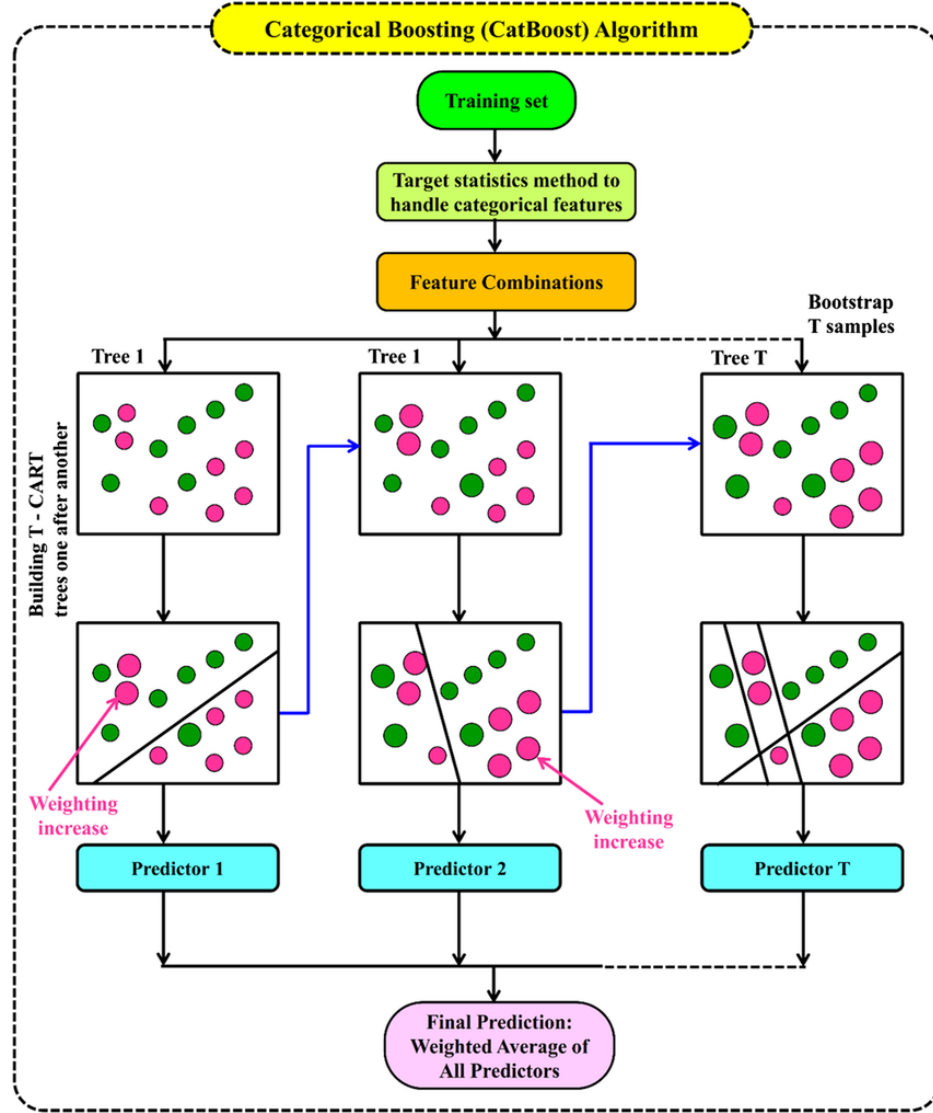


Figure 2.8: Representation of CatBoost Classifier [43]

eta parameter in the implementation [40].

2.3.1.6 CatBoost

CatBoost, which stands for Categorical Boosting, is an advanced implementation of gradient boosting designed to handle categorical variables optimally. Developed by researchers at Yandex, CatBoost is distinguished by its handling of categorical data without the need for extensive pre-processing that other machine learning algorithms might require [44]. CatBoost improves upon traditional boosting methods by intro-

ducing several innovations:

- **Ordered Boosting:** This technique helps prevent overfitting by using a different permutation of the training data for each boosting iteration, thus reducing bias and variance.
- **Symmetric Trees:** CatBoost builds only symmetric trees which ensures faster execution and reduces the risk of overfitting compared to asymmetric trees.

Fig. 2.8 represents the working of algorithm, it works by sequentially constructing decision trees, each correcting the errors of the preceding one. Each tree is built using the gradient of the loss function, which guides the learning process. The categorical features are transformed using a combination of one-hot encoding for low cardinality features and mean encoding for high cardinality features.

The predictive model $F(x)$ in CatBoost is formulated as follows:

$$F(x) = F_0(x) + \sum_{t=1}^T \eta \cdot f_t(x)$$

where $F_0(x)$ is the initial approximation, η is the learning rate, $f_t(x)$ is the contribution of the t -th tree, and T is the total number of trees.

To predict the probability of a binary outcome, CatBoost applies a transformation to the output of the ensemble of trees:

$$p(y = 1|x) = \frac{1}{1 + e^{-F(x)}}$$

where e is the base of the natural logarithm, ensuring that the output is scaled between 0 and 1.

Regularization techniques in CatBoost include the control of tree depth, l2 leaf value penalty, and the rate of learning rate decay, which are crucial for tuning the model to achieve better generalization on unseen data.

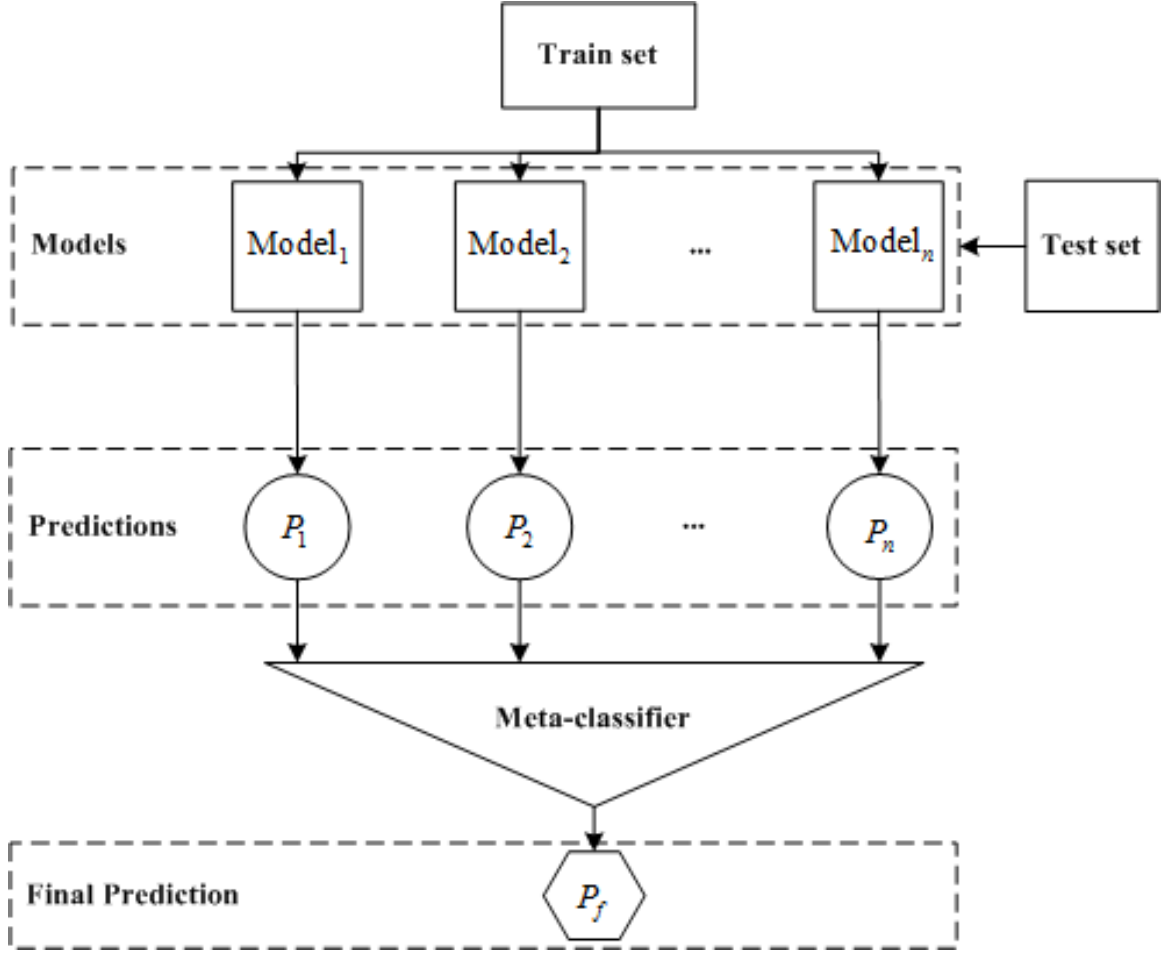


Figure 2.9: Representation of Stacking Method [47]

2.3.1.7 Stacking Methods

Stacking, also known as stacked generalization, is an ensemble learning technique that combines multiple classification models via a meta-classifier [45]. Unlike traditional ensemble methods such as bagging and boosting, which primarily focus on a single type of model, stacking involves multiple different classifiers; this approach is beneficial for complex classification tasks like intrusion detection in CAN bus networks, where different models might capture different aspects of the data patterns [46].

The primary objective of stacking is to leverage the diverse strengths and unique perspectives of each base model to improve the system's overall performance. Each model may excel in different areas or capture different patterns within the data, and stacking allows these strengths to be collectively utilized [48]. By doing so, stacking

can often achieve higher accuracy and robustness compared to any individual model alone.

The stacking process involves the following steps as shown in Fig. 2.9:

- **Training Base Models:** Multiple base models are trained on the training dataset. Each model provides its predictions on the test set. For this study, we used the following base models- KNN, Decision Trees, Random Forest, XGBoost and CatBoost.
- **Generating Predictions:** The predictions from each base model are collected.
- **Meta-classifier Training:** A meta-classifier is trained on the predictions made by the base models. This classifier learns how to combine the outputs of the base models to make a final prediction. For this study, we used Logistic regression as a meta-classifier.
- **Final Prediction:** The meta-classifier makes the final prediction based on the combined outputs from the base models.

2.3.2 Deep Learning Models

Deep Learning is a subset of machine learning, involves algorithms inspired by the human brain's structure, notably artificial neural networks. These advances algorithms mimics brain functionality, particularly how neurons interconnect and transmit information through the nervous system. In deep learning, models are constructed with multiple layers, typically requiring at least three layers of *input*, *hidden*, and output to function effectively. Each layer processes information received from the previous layer and passes it to the next. This layered architecture allows deep learning models to learn complex patterns in data, to perform highly sophisticated tasks, making them powerful tools for various applications.

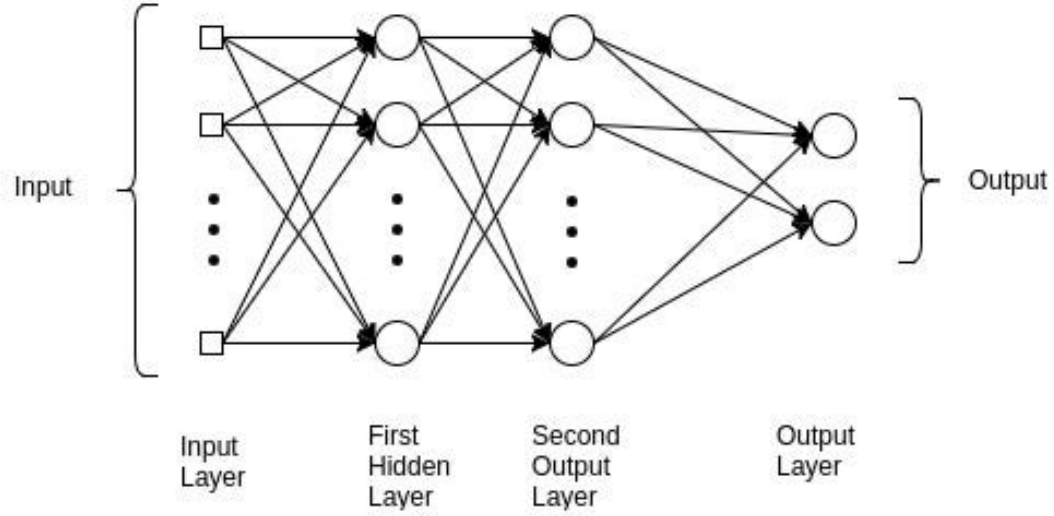


Figure 2.10: Representation of MLP Classifier [50]

2.3.2.1 Multi-Layer Perceptrons (MLP)

It is a type of feed forward neural network. Fig. 2.10 represents the MLP classifier, it consists of three types of layer, an input layer, one or more hidden layers, and an output layer [49]. The input layer receives the data, and each subsequent layer processes it and transforms it into something the output layer can use to perform classification [50]. The neurons in these layers apply a weighted sum to their inputs and follow this with a non-linear activation function.

The outputs of one layer are used as inputs to the next layer, leading to a forward propagation of data from the input to the output layer. Mathematically, the output z_i for each neuron is determined by the equation:

$$z_i = \sum_j (w_{ij} \cdot x_j) + b_i$$

where w_{ij} represents the weights, x_j the inputs, and b_i the bias associated with the i -th neuron. Following this, an activation function σ transforms this output to:

$$a_i = \sigma(z_i)$$

Common activation functions used in MLPs include sigmoid, ReLU, and tanh.

MLPs use backpropagation, a method where errors are propagated backwards through the network to adjust the weights, minimizing the difference between the predicted output and the actual output. The effectiveness of MLPs in generalizing from training data to unseen data is a critical measure of their performance and is influenced by factors such as the number of layers, neurons and the choice of activation function.

2.3.2.2 Long Short-Term Memory (LSTM)

It is a special kind of Recurrent Neural Network (RNN) capable of learning long-term dependencies in data sequences. Originally introduced by Hochreiter and Schmidhuber in 1997 [51], LSTMs are particularly useful for tasks where context from the input data is essential for making predictions, such as in time series analysis, natural language processing, and speech recognition.

Unlike standard feedforward neural networks, LSTMs have a chain-like structure with repeating modules of neural network layers, where each module is designed to allow information to persist. The core idea behind LSTMs is the cell state, which runs through the entire chain, with only slight linear interactions. This design helps it to remove or add information to the cell state, regulated by structures called gates as represented in Fig. 2.11. LSTM consists of following gates:

- Forget Gate: Decides what information is discarded from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Input Gate: Decides which values will be updated in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- Cell State Update: Creates a vector of new candidate values that could be

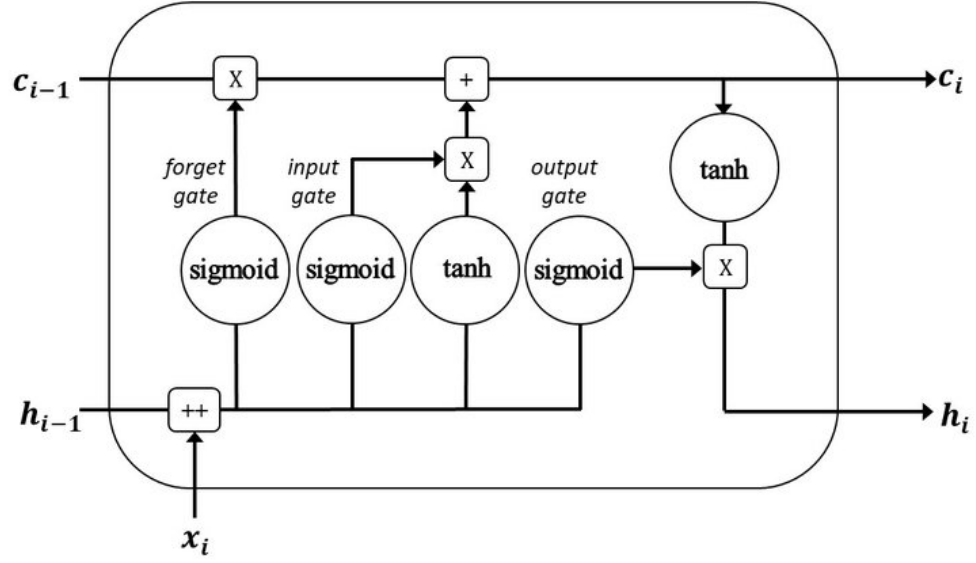


Figure 2.11: Representation of LSTM Memory Cell [51]

added to the state.

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- Output Gate: Decides what the next hidden state should be.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- Final Cell State and Output Update:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$h_t = o_t * \tanh(C_t)$$

The gates of LSTMs use the sigmoid function σ which outputs values between 0 and 1, describing the amount to which each component should be let through. The cell updates are regulated by pointwise multiplication operations $*$, which contribute to the network's ability to mitigate the vanishing gradient problem that affects many RNNs.

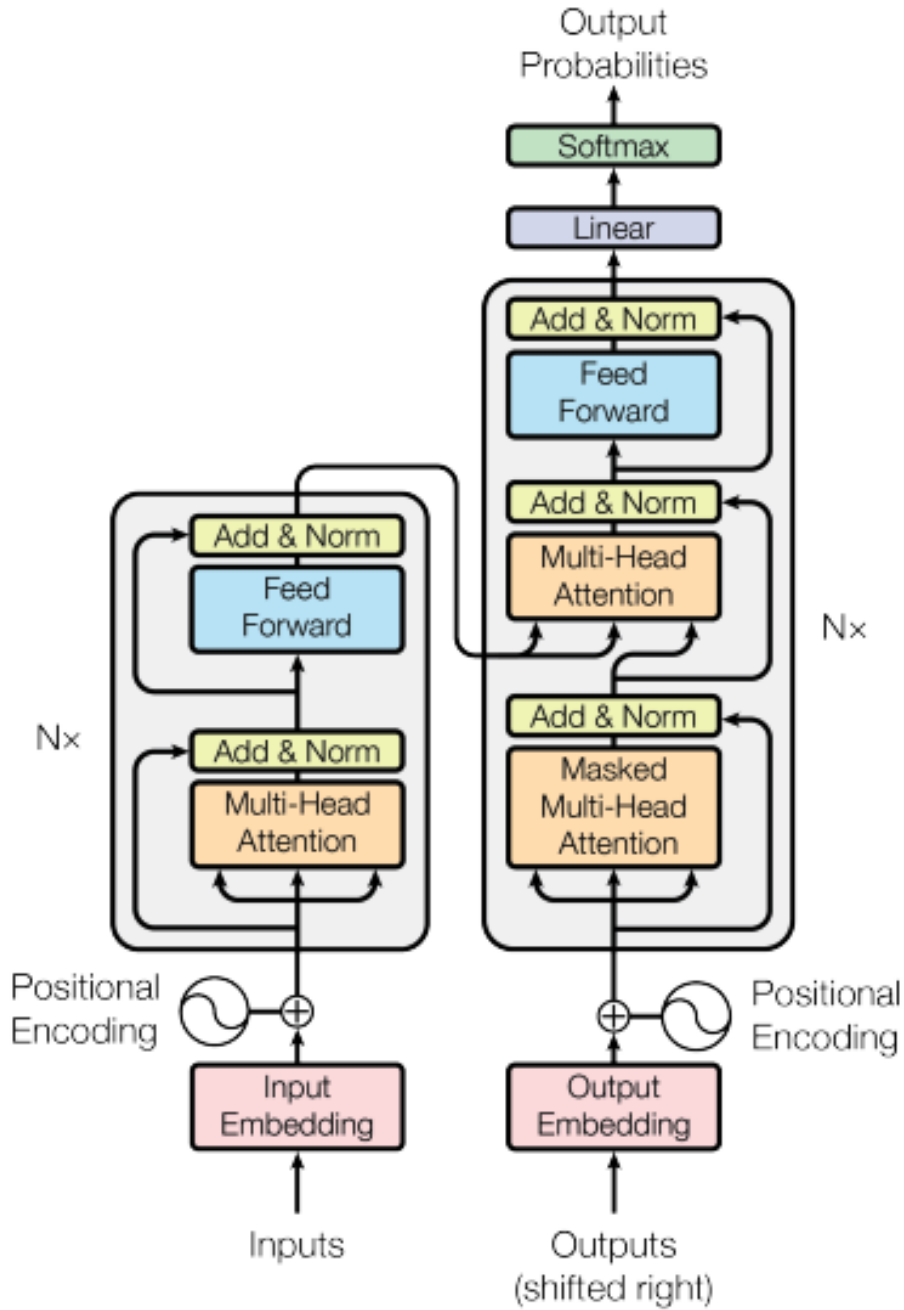


Figure 2.12: Transformer Architecture [52]

LSTMs have shown remarkable performance across a variety of applications. Their ability to model time series makes them ideal for attack prediction in In-vehicle networks.

2.3.3 Transformer-based Models

In recent years, Transformers have been popular and efficient in Natural Language Processing tasks, usually for time series data analysis. Transformers work in a bidirectional manner, and these are popular attention models for language modelling. As transformers work on the encoder-decoder mechanism in which text as an input is provided to the encoder and decoder outputs the possible prediction. A typical transformer model consists of six components: Embeddings, Positional Encoding, Multi-Headed Attention Layers, Feed-Forward Layers, Residual Connections, and Masks (see Fig. 2.12). The details of implementing a transformer based architecture can be read from this Github repo [53].

2.3.3.1 Bidirectional Encoder Representations from Transformers (BERT)

todo.

2.4 Literature Review

Securing vehicle communication is crucial as it directly impacts the safety of vehicles and the lives of their occupants, including drivers and passengers. Achieving this security is challenging with the CAN protocol due to its inherent limitations and vulnerabilities to various attacks such as DoS, impersonation, and fuzzy attacks. These vulnerabilities present a significant challenge and an opportunity for innovation in developing an IDS for such networks.

Several studies have been conducted to address the issue of securing vehicular communication. The most related studies that developed IDS for In-vehicle networks are discussed below, highlighting existing research gaps.

In [12], the authors presented a comprehensive review of IDS for connected and autonomous vehicles (CAVs) using machine learning techniques. This article begins by highlighting the main advantages of CAVs, such as improved vehicle safety and transport efficiency. It confirms their vulnerability to various attacks, including targeted ones, on vehicle-to-vehicle networks. This study aims to describe machine

learning-based IDS approaches, discuss publicly available datasets related to CAV security, and examine the present and future research aspects. The survey delves into intra-vehicle and inter-vehicle network security details, describing how machine learning can effectively detect and mitigate security threats. The authors provide a critical analysis of different ML models, the nature of cyber-attacks they can counter, and the effectiveness of these models in real-world scenarios. Furthermore, the paper reviews the datasets available for training and testing these systems and examines the simulation environments and testbeds used for developing IDS solutions in CAVs.

The work in [54] presents a detailed examination of various pivotal datasets for enhancing security within vehicular networks. It explores the significance of these datasets in developing and testing artificial intelligence solutions to counter security threats prevalent in intra-vehicle and inter-vehicle communications. By introducing an innovative taxonomy, the study categorizes and evaluates these datasets based on their structure, application, and the types of threats they address. This comprehensive survey offers insights and guidelines on selecting the most appropriate datasets for specific security studies in vehicular environments, thus aiding in developing robust security measures to counter the unique challenges of vehicular network security.

Aliwa et al.[4] provide a comprehensive review of the vulnerabilities in the communication protocols used within vehicles, particularly the CAN Bus, which lacks built-in security measures and is susceptible to various types of attacks. They emphasize the increased risks associated with Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) communications. The paper outlines the standard serial protocols like CAN, Local Interconnect Network (LIN), and FlexRay, discussing their usage in vehicles and security shortcomings. The discussion extends to various attacks experienced in real-world scenarios, such as DoS and bus injection attacks, highlighting the need for robust security mechanisms. The authors categorize and compare various vehicular cybersecurity approaches, focusing on encryption, authentication, and IDS, as well as on-demand constraints and performance under real-time constraints. This study assessed current solutions, their integration into simulated environments and their validation against standard attack methods. The paper concludes with poten-

tial research avenues, emphasizing the need to develop high mitigation strategies and the challenges that need to be addressed to improve the security of transportation networks.

The authors of [13] provide an in-depth review of advanced machine learning and deep learning techniques applied to intrusion detection systems within intelligent vehicle networks. The paper emphasizes the vulnerabilities in the CAN protocol, commonly used in IVNs, which lack inherent security features. To mitigate the growing cybersecurity risks associated with intelligent and interconnected vehicles, the authors propose a taxonomy of IDS methodologies, categorizing them into conventional ML, DL, and hybrid approaches. These methods are designed to detect and prevent attacks, such as spoofing, denial of service, and eavesdropping. The paper’s key highlight is the transition from traditional signature-based IDS methods to anomaly-based detection approaches. The latter offers advantages in identifying unknown or novel cyber threats, making AI-driven techniques critical in detecting complex attacks. The paper reviews DL models such as Recurrent neural networks, Convolutional Neural Networks (CNN), and recent approaches like transformers and federated learning. Emerging technologies like transfer learning and collaborative IDS frameworks are also explored, and they aim to improve the adaptability and robustness of IDS solutions. Despite the promising advancements, the authors note several limitations in the current approaches. While effective in detecting known threats, many methods need help with generalizability to new attack scenarios or real-time detection in resource-constrained environments, such as those in-vehicle ECUs. Additionally, the paper emphasizes that future research should focus on improving real-time data analysis and integrating cross-layer security measures to ensure that the increasing connectivity of vehicles does not compromise their safety. This survey serves as a comprehensive guide for researchers and practitioners looking to enhance security in intelligent vehicle networks, providing insights into the current state of ML/DL-based IDS and identifying potential research gaps.

The authors in [55] presented a deep learning-based solution to enhance the security of CAN bus systems in modern vehicles. Due to the critical nature of the

CAN bus in facilitating communication among various ECUs in vehicles, its security is paramount. However, the standard CAN protocol lacks encryption and authentication, rendering it susceptible to several attacks such as DoS, Fuzzing, and Spoofing. To address these vulnerabilities, the authors propose a CNN-based IDS that effectively detects and mitigates such attacks. They developed and trained their model using data collected from three different vehicles that includes different attacks, demonstrating a detection accuracy of up to 99.99 percent, which signifies the model’s efficacy in identifying malicious activities within the CAN bus system. The paper thoroughly details the types of attacks the IDS can detect, emphasizing the performance of the CNN model across various simulated intrusion scenarios. This IDS detects known attack types and holds the potential for future adaptation to recognize new threats. The only problem with this work is that the dataset is used to train models is unbalanced, has a higher proportion of benign traffic. And the proposed IDS has not been tested in real-world driving conditions. Overall, the study contributes significantly to providing a scalable and effective solution to enhance the resilience of in-vehicle networks against increasingly sophisticated threats.

In [14], the study focuses on enhancing the security of CAN bus systems in vehicles using a LSTM-based IDS. The authors emphasize the vulnerability of CAN systems, which lack inherent security features like encryption and authentication, making them susceptible to attacks. To address this vulnerability, they developed an LSTM model to identify malicious activities within the CAN bus. They generated a dataset by simulating various attacks on a real vehicle’s CAN system. The LSTM model was trained with this data, consisting of benign and attack instances, achieving a remarkable detection accuracy of 99.9949 percent with low false positive and negative rates. This high level of accuracy demonstrates the usefulness of the LSTM model in distinguishing between normal operations and potential security breaches in the CAN bus system. But the drawback of this study is that the approach was not tested in real-time scenarios.

In [56], they introduce CANet, a novel unsupervised deep learning architecture for intrusion detection on automobile CAN bus data. CANet is designed to handle

the unique data structure of CAN bus traffic, where different message types with varying numbers of signals are sent at different times. The architecture consists of individual LSTM input models for each message ID, which capture the temporal dynamics of corresponding signals. These inputs are then aggregated into a joint latent vector and fed into a fully connected autoencoder subnetwork, allowing the system to learn interdependencies between signals from all IDs. The authors evaluate CANet on CAN data, demonstrating that it outperforms previous machine learning-based methods for intrusion detection. The network can detect known and unknown attack scenarios by identifying signals that deviate from normal behaviour or break physical relationships between signals. However, CANet has several drawbacks. Its reliance on LSTMs for each message ID and the use of a fully connected autoencoder make the model computationally expensive, potentially limiting its deployment in resource-constrained environments like vehicle ECUs. Moreover, the model’s complexity makes it challenging to interpret, which could be a disadvantage in situations requiring explainability. CANet also exhibits latency in detecting subtle or slowly evolving attacks, such as continuous change scenarios, and its ability to detect rare attacks, like flooding, may require complementary rule-based systems. Despite these challenges, CANet is a significant advancement in In-vehicle intrusion detection, offering robust detection capabilities and the potential for identifying unknown attack types unsupervised.

In [15], Javed et al. addresses the critical vulnerabilities present in the CAN protocol, such as weak access control, lack of message authentication, and the absence of encryption, which make it susceptible to various cyberattacks. To mitigate these risks, the authors propose CANintelliIDS, an intrusion detection system combining CNN with an attention-based Gated Recurrent Unit to detect both single and mixed intrusion attacks on the CAN bus. This hybrid approach leverages the CNN’s ability to extract significant features from CAN data sequences while the GRU captures temporal dependencies, enhancing the detection of complex attack patterns. The system was tested using CAN bus data, including various attack types such as DoS and fuzzy attacks, demonstrating a 10.79% performance improvement over existing

methodologies. The results showcased higher accuracy and F1 scores, especially in balanced and unbalanced datasets, highlighting the model’s robustness across diverse scenarios. CANIntelliIDS not only detects individual attacks but also performs well in identifying mixed intrusion attacks, which are combinations of multiple attack types. While the system showed significant improvements, some limitations exist. The complexity of the hybrid CNN-GRU architecture increases computational overhead, which may limit its practical deployment in resource-constrained environments. Additionally, the model’s performance on real-time attack detection requires further exploration to ensure its efficacy in live vehicular networks. Overall, CANIntelliIDS makes a noteworthy contribution to the field of in-vehicle IDS by addressing critical security challenges and demonstrating robust detection capabilities against various attack types. Future work should optimize the model for real-time performance and reduce computational demands to make it more practical for automotive applications.

In [24], OTIDS is presented as a novel intrusion detection system designed for In-vehicle CAN systems by leveraging the analysis of offset ratios and time intervals between request and response messages. The study focuses on the vulnerabilities of the CAN protocol, notably the lack of source and destination address validation, which allows attackers to inject malicious messages easily. The proposed system does not require modifications to the existing CAN protocol, making it a practical solution for current vehicles. The intrusion detection method in OTIDS relies on monitoring the offset ratios and time intervals between transmitted remote frames and their corresponding responses. Under normal conditions, each CAN node responds with a fixed offset ratio and time interval. However, these values deviate significantly during attacks, providing a reliable means of detecting intrusions. The authors demonstrate the system’s effectiveness in detecting various attacks, such as Denial of Service and Fuzzy attacks, which exploit the inherent weaknesses of the CAN protocol. In attack-free conditions, response times and offset ratios remain consistent, but these parameters degrade during an attack. One key advantage of OTIDS is its ability to detect compromised nodes and provide a comprehensive dataset for further research on CAN security. The system achieves high detection accuracy with minimal

computational overhead, making it suitable for real-time applications in connected vehicles. This study makes an essential contribution to enhancing vehicle communication security without needing fundamental changes to the CAN infrastructure, offering practical protection against emerging threats in modern connected vehicles. Despite its strengths, OTIDS may have limitations in dealing with highly complex, coordinated attack patterns that manipulate request and response sequences more sophisticatedly. Further work could focus on refining the detection thresholds and exploring its adaptability to newer communication protocols beyond CAN, such as FlexRay or Ethernet.

The authors in [16] introduce CANLite, a lightweight system designed to enhance security in modern vehicle networks. CAN, a commonly used protocol for communication between vehicle components, lacks robust security features, leaving it vulnerable to cyberattacks. Unlike traditional methods that focus on individual sensors and often miss subtle threats, CANLite uses multitask learning to analyze relationships between multiple sensors, enabling the detection of stealthy attacks that might otherwise go unnoticed. This approach significantly reduces computational load, making it suitable for resource-constrained environments like automotive systems. With its LSTM-based architecture, CANLite captures time-based patterns in data and predicts potential threats in real time. Tests on real-world datasets show that CANLite reduces memory usage by 50% while maintaining high detection accuracy against attacks like DoS and fuzzy attacks, and it can even pinpoint the source of the anomaly for quicker response. However, some limitations include its focus on known attack patterns, which may reduce effectiveness against entirely novel threats. Additionally, real-time performance in highly complex attack scenarios may still pose a challenge, and the system may require further optimization for deployment in highly dynamic or large-scale vehicle networks. Overall, CANLite offers a practical solution for vehicle network security without overwhelming system resources.

A smart way to detect cyberattacks on vehicle networks is proposed in [57] by monitoring the timing of CAN messages. As modern vehicles become more connected to external networks, they are increasingly vulnerable to attacks like message

injection and hijacking essential vehicle functions. The method works by analyzing the time intervals between CAN messages, known as event-triggered intervals, and using statistical techniques to spot any unusual patterns that could indicate an attack. Tested on CAN data, the system effectively identified attacks like flooding, fuzzy, malfunction, and replay attacks with high accuracy—up to 99% for detecting anomalies and 94% for identifying specific attack types when using tree-based machine learning models. One key advantage is that it does not need prior knowledge of specific CAN IDs, making it adaptable to many vehicles. However, the system has some limitations. It relies on predefined attack scenarios, which may only catch partially new or evolving attack strategies. Additionally, while the method efficiently detects certain types of attacks, its performance under more dynamic and complex attack conditions is yet to be thoroughly evaluated. Despite these challenges, this approach offers a practical and effective way to improve vehicle cybersecurity without requiring significant changes to existing CAN protocols.

TENET[58] proposed as a novel framework designed to detect anomalies in-vehicle networks using a temporal convolutional neural network (TCN) with an integrated attention mechanism. As vehicles become more interconnected and reliant on ECUs, the risk of cyberattacks grows. TENET addresses these risks by learning the long-term dependencies between messages in the vehicle’s network and identifying deviations that may indicate an attack. By leveraging the attention mechanism, TENET detects subtle and complex attacks that might evade traditional methods. Tested on real-world datasets, TENET outperforms previous methods, achieving a 32.7% improvement in false negative rate and a 19.14% boost in Mathews Correlation Coefficient while also reducing model parameters by 94.62%, making it highly efficient for deployment in resource-constrained environments like vehicle ECUs. But TENET also has some drawbacks. While it excels in detecting known attacks, its performance on entirely new, sophisticated attack patterns remains an area for further research. Additionally, though the attention mechanism reduces overhead, some computational complexity still poses challenges for real-time deployment in more complex vehicle systems. Despite these challenges, TENET represents a significant advancement in automotive

cybersecurity, offering a practical and scalable solution for anomaly detection.

The authors in [59] present a cutting-edge approach for detecting cyberattacks in-vehicle networks using a bidirectional Generative Pre-trained Transformer (GPT) model. The authors leverage the GPT model’s ability to detect subtle patterns in CAN ID sequences by treating these sequences like sentences composed of individual CAN IDs. This approach allows the system to predict anomalies by comparing expected and attack patterns in the data. The unique advantage of this system is its bidirectional nature, meaning it can analyze both past and future CAN IDs simultaneously, improving the detection of attacks with minimal deviations from normal behaviour. However, limitations include the model’s dependency on high-quality training data, which might not be available in every scenario. The complexity of the bidirectional GPT model also introduces significant computational overhead, which could be a challenge for deployment in resource-constrained ECUs. Despite these challenges, Bi-GPTIDS offers a powerful and efficient method for detecting subtle and complex cyberattacks in automotive networks.

CHAPTER 3

Methodology

This chapter describes our proposed framework, including tools and libraries used to implement our framework (HyCAN), system configuration, hyperparameters for training, dataset details, details of evaluation metrics and different research scenarios used to evaluate our framework.

3.1 Tools and Libraries

For the implementation of the methodology and execution of the experiments, we utilized the following technical resources and tools:

- **Programming Language:** Python 3
- **IDE:** Microsoft Visual Studio Code
- **Jupyter Notebook:** Google Colaboratory Pro
- **GPU:** Available through Google Colaboratory (Nvidia A100)
- **RAM:** High-RAM configuration (40 GB)
- **Python Libraries:**
 - Pandas - for data manipulation and analysis.
 - Scikit-learn - for machine learning algorithms and data processing.
 - NumPy - for numerical computing.
 - Seaborn - for statistical data visualization.

- Matplotlib - for creating static, interactive, and animated visualizations in Python.
- PySpark - for processing large-scale data.
- PyTorch - for machine learning and deep learning applications.
- Transformers - for using pre-trained models.

This setup provided a robust platform for developing and testing our models, facilitating efficient data handling, computation, and visualization capabilities crucial for our research.

3.2 System Configuration

We designed, build the proposed architecture and compared all the models against each other. All the experiments are conducted on Macbook-Air 2020 with device specifications of a Apple M1 chip with 8 core CPU and 7 core GPU, 8 GB RAM, and Mac OS Sonoma version 14.5.1.

3.3 Dataset

Our work utilized the recently published and publicly accessible *can-train-and-test* dataset [17] which was collected from on-road data involving four different vehicles and six different drivers to create a diverse CAN dataset for machine learning applications. The dataset was collected utilizing Korlan USB2CAN cable. This cable connects to the OBD-II port and converts the raw OBD-II data into USB data. This conversion enables seamless communication with the OBD-II port using Linux’s SocketCAN subsystem and can-utils utilities, which are essential for capturing and processing CAN bus data.

The official Github repository [17] of the dataset contains multiple folders specifically designed to develop a different types of IDS. For this study, we used can-ml repository, which contains labelled attack-free and attack samples, well-defined for

building ML-based IDS. The dataset consists of nine different types of unique attacks, but for this work, we only considered five types of different attacks mentioned below:

- **DoS:** Attack data is obtained by overwhelming the network with excessive messages to prevent legitimate communications.
- **Fuzzy:** Random CAN messages are injected into the network, disrupting the normal operation of vehicle systems.
- **Gear Spoofing:** The falsified messages were sent to manipulate the vehicle's gear control system.
- **Speed Spoofing:** The vehicle's speedometer is the target that sends fraudulent speed data to the display.
- **Standstill:** Signals are sent that make the vehicle's systems believe it is not moving, even when it is in motion.

Table 3.1 represents four key attributes of the can-train-and-test dataset, highlighting its comprehensive nature and the variety of data it encompasses. The dataset's diversity is critical for developing robust machine learning models capable of accurately detecting and classifying different types of cyberattacks on In-vehicle networks.

Table 3.1: The Attributes of can-train-and-test

Attribute	Explanation
timestamp	recorded time (s)
arbitration_id	identifier of CAN message in HEX (ex. 043f)
data_field	data value (byte)
attack	1 or 0, 1 represents injected message while 0 represents normal message

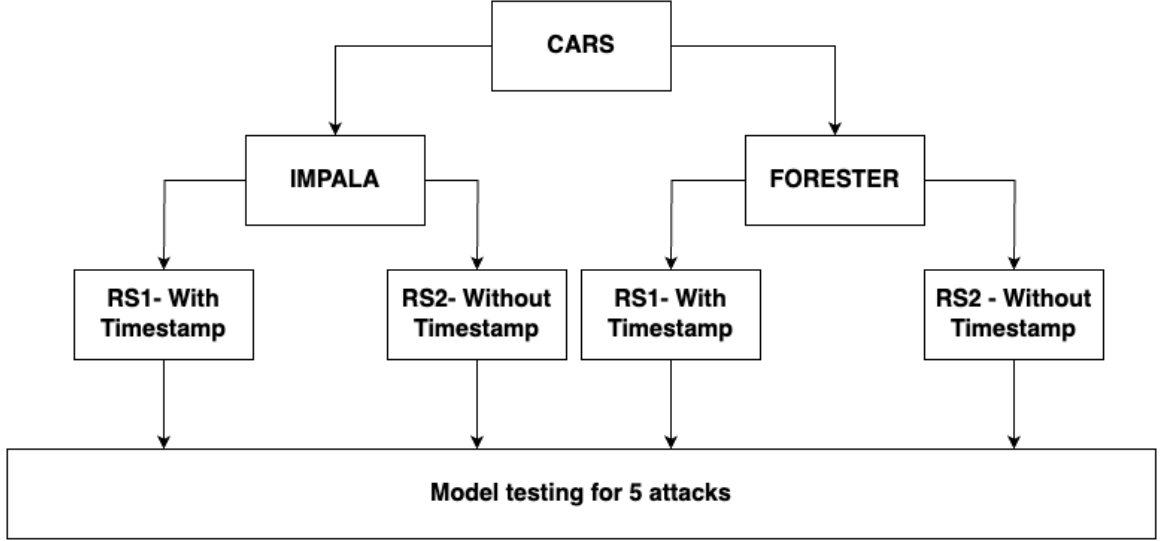


Figure 3.1: Research Scenarios

3.4 Research Scenarios

After conducting an extensive literature review, we observed that most In-vehicle networks attacks typically involve either manipulating the arbitration ID or altering data field values within the CAN frame. This observation has led us to question the relevance of the timestamp feature when training and testing various models within our framework. Consequently, this has introduced an additional research question to our study: determining the significance of the timestamp feature in our models.

Therefore, to evaluate the effectiveness of our proposed framework and understand the impact of the timestamp feature in CAN bus data, we have established two specific research scenarios *i.e* RS1 and RS2, for two vehicle models: the Impala and the Forester, from the dataset. Fig. 3.1 represents scenarios designed to investigate the influence of the timestamp on model performance in detecting intrusions:

3.4.0.1 RS1 - With Timestamp

This scenario utilizes the complete CAN dataset, including the timestamp feature. The inclusion of timestamps aims to explore whether temporal information contributes to the model's ability to detect and differentiate between normal operations and various attack patterns.

3.4.0.2 RS2 - Without Timestamp

In contrast, this scenario omits the timestamp from the dataset to determine the model’s effectiveness based solely on the arbitration ID and data fields. This helps assess whether the absence of temporal data impacts the accuracy and detection capabilities of our models.

Both scenarios are applied to the Impala and Forester vehicles, and each model is tested against five distinct types of attacks. This structured approach allows us to compare and contrast the performance of various models under varied conditions, providing comprehensive insights into the robustness of the models. The findings from these scenarios help answers the critical research question regarding the importance of timestamps in modelling attack detection on In-vehicle networks.

3.5 Proposed Methodology

In order to develop the HyCAN, a variety of ML approaches have been implemented using Python. Fig. 3.2 shows the overall architecture of the proposed framework.

The proposed framework for for performance study of CAN bus intrusion detection leverages both traditional machine learning algorithms and advanced transformer-based models. The architecture is divided into distinct stages: *Data Preprocessing*, *Model Construction*, and *Model Evaluation*. Each stage is carefully designed and discussed in detail, so that it is easier for anyone to re-implement the whole approach.

3.5.1 Data Preprocessing

It is a crucial step in preparing the CAN data for modelling. This stage ensures the data is clean, balanced, and appropriately formatted for the selected model type. It includes a series of steps discussed below:

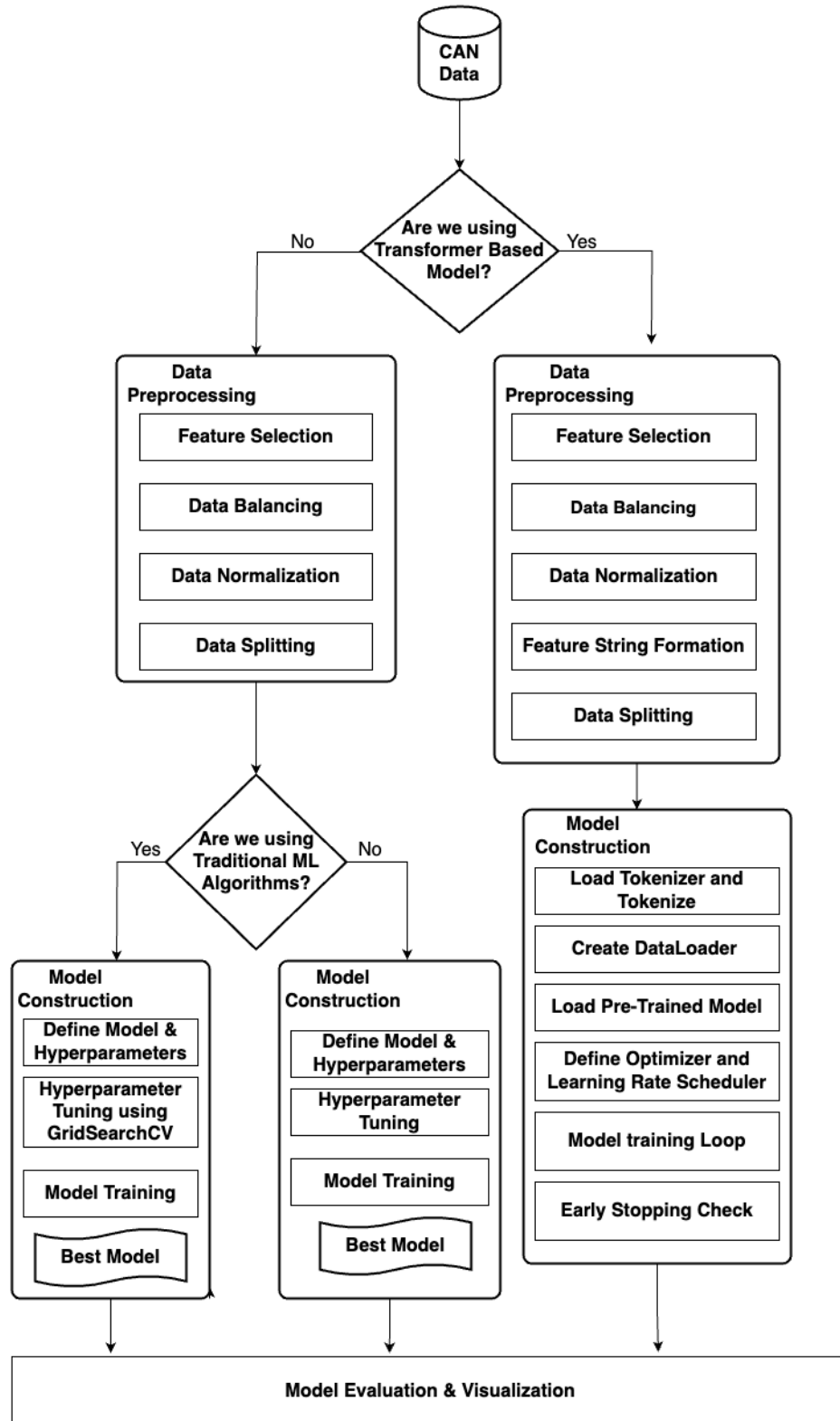


Figure 3.2: Proposed Framework

3.5.1.1 Feature Selection

This is the first step included in the data preprocessing step. As illustrated in Table 3.1, the dataset contains only three features- *timestamp*, *arbitration-id*, *data-field*. So, in this step, we selected the features based on the research scenario, *i.e.*, in RS1- we included the timestamp, but not in RS2.

3.5.1.2 Data Balancing

As the *can-train-test* dataset [17] is diverse and extensive (around 50GB), it contains millions of samples for each attack type including attack-free state. Due to limited computational resources we took only 100k samples for each attack type, that’s how we balanced the data.

3.5.1.3 Data Normalization

In the dataset, all the features except timestamp contains hexadecimal values, which can affect the performance of machine learning algorithms. So we used data normalization to convert all values to decimal and then scaled the features to a standard range (e.g., between 0 and 1 or using z-score normalization). This step ensures that the model treats all features equally and improves convergence during training.

3.5.1.4 Feature String Formation

An additional preprocessing step is included for transformer-based models, as they typically require input data in a sequential or textual format. In the *can-train-test* dataset, each entry after normalization contains a numerical value which cannot be sent as input to BERT. Therefore, we selected and combined all features to form a feature string representation of each entry. A separator “/” was added between the features to maintain the feature integrity and avoid arbitrary tokenization during the BERT tokenization process.

For example, when two adjacent features have values of 28 and 29, the “/” separator results in the feature string “28/29” being used and split into three tokens:

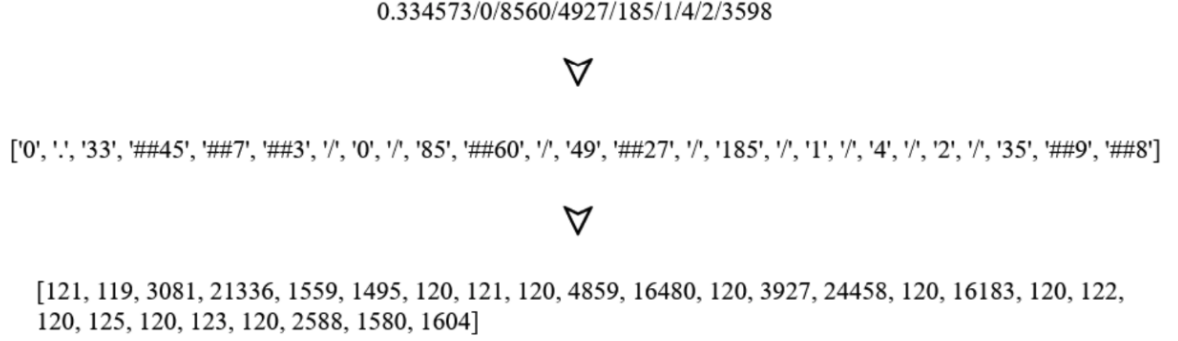


Figure 3.3: Illustration of BERT Tokenization on random numerical feature string

“28”, “/”, and “29”. Without the separator, “2829” could be split into tokens like “2829”, “282” and “9”, or “2” and “829”. Therefore, the “/” separator helps the model distinguish individual features during tokenization. Fig. 3.3 represents the illustration of BERT tokenization process on a random numerical feature string.

3.5.1.5 Data Splitting

In this step, the preprocessed data is split into training, validation, and testing data. We used the training set to train the model, the validation set for hyper-parameter tuning and best model selection, and the testing set to evaluate the final selected model’s performance. A typical split ratio for these sets is 70:15:15 respectively.

3.5.2 Model Construction

The development of HyCAN for the performance comparative study is approached with a key consideration: extensibility. The framework is designed to be modular, allowing for easy integration of new models and functionalities in the future. Each model implemented within the framework resides in its own module, facilitating maintenance, updates, and enhancements. Here’s an extensive overview of the model construction process for traditional machine learning algorithms, deep learning architectures and transformer-based models discussed in detail with the pseudo-code listing:

Model	Hyperparameter(s)	Values/Settings
KNN	<code>n_neighbors</code>	{3, 5, 7, 9}
	<code>weights</code>	{uniform, distance}
	<code>metric</code>	{euclidean, manhattan}
LogisticRegression	<code>C</code>	{0.01, 0.1, 1, 10, 100}
	<code>solver</code>	{lbfgs, liblinear}
DecisionTree	<code>criterion</code>	{gini, entropy}
	<code>max_depth</code>	{None, 10, 20, 30, 40, 50}
RandomForest	<code>n_estimators</code>	{10, 50, 100, 200}
	<code>criterion</code>	{gini, entropy}
	<code>max_depth</code>	{None, 10, 20, 30, 40, 50}
XGBoost	<code>n_estimators</code>	{50, 100, 200}
	<code>learning_rate</code>	{0.01, 0.1, 0.2}
	<code>max_depth</code>	{3, 5, 7, 9}
CatBoost	<code>iterations</code>	{100, 200, 300}
	<code>learning_rate</code>	{0.01, 0.1, 0.2}
	<code>depth</code>	{4, 6, 8}
MLP	<code>hidden_layer_sizes</code>	{(50,), (100,), (50,50), (100,50)}
	<code>learning_rate</code>	{0.01, 0.001}
	<code>optimizer</code>	Adam
	<code>criterion</code>	BCEWithLogitsLoss
	<code>epochs</code>	50
	<code>early_stopping</code>	Patience = 5
BERT	<code>max_length</code>	128
	<code>batch_size_train</code>	32
	<code>batch_size_val</code>	64
	<code>learning_rate</code>	2e-5
	<code>epochs</code>	10
	<code>optimizer</code>	AdamW
	<code>scheduler</code>	linear.schedule.with.warmup
	<code>early_stopping</code>	Patience = 2
LSTM	<code>hidden_dim</code>	{128, 256}
	<code>n_layers</code>	{1, 2}
	<code>bidirectional</code>	{True, False}
	<code>dropout</code>	{0.2, 0.3}
	<code>learning_rate</code>	{0.001, 0.0001}
	<code>batch_size</code>	32
	<code>epochs</code>	10
	<code>optimizer</code>	Adam
	<code>criterion</code>	CrossEntropyLoss
	<code>early_stopping</code>	Patience = 2

Figure 3.4: Tested Hyperparameters

3.5.2.1 Machine Learning Models

When using traditional machine learning algorithms, the model construction involves the following steps (see Listing 1 for psuedo-code):

- i. **Model Definition:** Once the data preprocessing is done, then this step involves selecting the appropriate machine learning algorithm (e.g. LR, KNN, DT, RF, XGBoost, CatBoost) and defining the model structure.

```

1  Machine Learning Models
2  Inputs:  Dataset, RS1, RS2, Models with Hyperparameters
3  Returns: Best Model Evaluation Metrics
4  Load dataset
5  # Preprocess dataset
6  if using RS1:
7      Convert timestamp for RS1
8  else if using RS2:
9      Remove timestamp for RS2
10 Convert arbitration_id and data_field to appropriate formats
11 # Extract features and labels
12 if using RS1:
13     X ← [datetime, arbitration_id, data_field]
14     y ← attack
15 else if using RS2:
16     X ← [arbitration_id, data_field]
17     y ← attack
18 # Split data into training and test sets
19 X_train, X_test, y_train, y_test ← train_test_split(X, y,
    test_size=0.2, random_state=42)
20 # Scale features
21 Scale X_train and X_test using MinMaxScaler
22 # Define models and their hyperparameters
23 models ← {
24     'KNN': {hyperparameters},
25     'LogisticRegression': {hyperparameters},
26     'DecisionTree': {hyperparameters},
27     'RandomForest': {hyperparameters},
28     'XGBoost': {hyperparameters},
29     'CatBoost': {hyperparameters} }
30 # Perform GridSearchCV with K-Fold cross-validation
31 for each model in models:
32     Perform GridSearchCV to find best hyperparameters
33     Train model on training data
34     Evaluate model on test data
35     Record metrics (accuracy, precision, recall, F1 score, ROC
    AUC)
36     Print results and plot confusion matrix, ROC curve, and
    metrics bar plot
37 # Print summary of results for all models
38 # Define base models for stacking
39 base_models ← [
40     (KNeighborsClassifier), (DecisionTreeClassifier),
41     (RandomForestClassifier), (XGBClassifier), (CatBoostClassifier)
    ]
42 # Define meta-learner
43 meta_learner ← LogisticRegression()
44 # Define stacking classifier
45 stacked_model ← StackingClassifier(base_models, meta_learner)
46 # Train stacking model
47 Train stacked model on training data
48 # Predict and calculate metrics for stacked model
49 Predict using stacked model on test data

```

Listing 3.1: ML Models Pseudo-code

- ii. **Hyperparameter Tuning:** Hyperparameters are the parameters that govern the training process and structure of the machine learning models. Tuning these parameters is crucial for optimizing model performance. For this research, techniques such as GridSearchCV are used to systematically search for the best combination of hyperparameters. Fig. 3.4 represents tested hyperparameters for all models..
- iii. **Model Training:** Once the model and hyperparameters are defined, the model is trained on the training dataset. During this phase, the model learns the underlying patterns and relationships in the data to make accurate predictions.
- iv. **Best Model Selection:** After training, the model that performs best on validation data (based on defined performance metrics) is selected as the final model.

3.5.2.2 Deep Learning Models

When using deep learning architectures, the model construction involves the following steps (see Listing 3.2 for MLP and Listing 3.3 for LSTM psuedo-code):

- i. **Model Definition:** This step involves defining the architecture of the deep learning model, i.e. selecting the neural network architecture, either MLP or LSTM.
- ii. **Hyperparameter Tuning:** Hyperparameters are critical to the performance of deep learning models. This step involves experimenting with different combinations of hyperparameters such as the number of hidden layers, number of units in each layer, learning rate, dropout rate, and batch size. For this research, techniques such as GridSearchCV are used to systematically search for the best combination of hyperparameters . Fig. 3.4 provides hyperparameter configurations for LSTM and MLP models.
- iii. **Model Training:** Once the model architecture and hyperparameters are defined, the model is trained on the training dataset. The training process includes:

```

1  MLP Model for CAN Bus Intrusion Detection
2  Inputs:  Dataset, Hyperparameters, RS1, RS2
3  Returns: Best Model Evaluation Metrics
4  Load dataset
5  # Preprocess dataset
6  if using RS1:
7      Convert timestamp for RS1
8  else if using RS2:
9      Remove timestamp for RS2
10 Convert arbitration_id and data_field to appropriate formats
11 # Extract features and labels
12 if using RS1:
13     X ← [datetime, arbitration_id, data_field]
14     y ← attack
15 else if using RS2:
16     X ← [arbitration_id, data_field]
17     y ← attack
18 # Scale features
19 Scale X using MinMaxScaler
20 # Dataset preparation for PyTorch
21 Define 'CANBusDataset' class to convert features and labels into
    PyTorch tensors
22 Create 'train_dataset' and 'test_dataset' using 'CANBusDataset'
23 Create 'train_loader' and 'test_loader' using DataLoader with '
    batch_size=64'
24 # Define MLP model
25 Define 'MLP' class with:
26     input_size ← 3
27     hidden_layers ← hidden_layer_sizes
28     output_size ← 1
29     dropout_rate ← 0.5
30 # Train the model
31 Define 'train_model' function with:
32     Inputs: model, train_loader, criterion, optimizer, num_epochs
        , val_loader, patience
33     Training loop:
34         For each epoch:
35             Perform forward and backward propagation
36             Calculate and store training loss
37             Evaluate on validation set (if provided)
38             Implement early stopping based on validation loss
39 # Hyperparameter tuning
40 For each combination of hidden_layer_sizes and learning_rates:
41     Initialize MLP model
42     Train model using train_loader and validate using test_loader
43     Evaluate model on test set
44     Track the best model based on accuracy
45 # Evaluate the best model
46 Define 'evaluate_model' function to:
47     Perform model evaluation on the test set
48     Calculate metrics: Accuracy, Precision, Recall, F1 Score, ROC
        AUC

```

Listing 3.2: MLP Model Pseudo-code

```

1 LSTM Model for Intrusion Detection
2 Inputs: Dataset, RS1, RS2
3 Returns: Best Model Evaluation Metrics
4 Load dataset
5 # Preprocess dataset
6 if using RS1:
7     Convert timestamp for RS1
8 else if using RS2:
9     Remove timestamp for RS2
10 Convert arbitration_id and data_field to appropriate formats
11 # Extract features and labels
12 if using RS1:
13     X ← [datetime, arbitration_id, data_field]
14     y ← attack
15 else if using RS2:
16     X ← [arbitration_id, data_field]
17     y ← attack
18 # Scale features
19 Scale X using MinMaxScaler
20 # Reshape features for LSTM
21 X ← X.reshape(X.shape[0], 1, X.shape[1])
22 # Split data into training, validation, and test sets
23 X_train, X_test, y_train, y_test ← train_test_split(X, y,
    test_size=0.2, random_state=42)
24 X_train, X_val, y_train, y_val ← train_test_split(X_train,
    y_train, test_size=0.25, random_state=42)
25 # Convert to PyTorch tensors
26 Convert X_train, X_val, X_test, y_train, y_val, y_test to PyTorch
    tensors
27 # Create DataLoader
28 Create DataLoader for training, validation, and test sets with
    batch_size=32
29 # Define LSTM model
30 Define LSTM model with input_dim, hidden_dim, output_dim,
    n_layers, bidirectional, dropout
31 # Hyperparameter tuning
32 for each combination of hidden_dim, n_layers, bidirectional,
    dropout, lr:
33     Train LSTM model
34     Evaluate on validation set
35     Save the best model based on validation loss
36 # Load the best model
37 Load the best saved model
38 # Evaluate on the test set
39 Evaluate the best model on the test set
40 Calculate metrics(accuracy, precision, recall, F1 score, ROC AUC)
41 # Plot results
42 Plot confusion matrix, ROC curve, and metrics bar plot

```

Listing 3.3: LSTM Model Pseudo-code

- Forward propagation: The input data is passed through the network to obtain the output.
 - Loss computation: The difference between the predicted output and the actual output is measured using a loss function.
 - Backpropagation: The gradients of the loss with respect to the model parameters are calculated and used to update the parameters to minimize the loss.
 - Optimization: An optimizer like Adam is used to update the model parameters iteratively to minimize the loss.
- iv. Early Stopping: To prevent overfitting, early stopping is implemented by monitoring the model’s performance on the validation data. Training is stopped if the validation performance does not improve after a certain number of epochs. The model that performs best on the validation data is selected as the final model.
- v. Best Model Selection and Evaluation: After training, the model that performs best on validation data (based on defined performance metrics) is selected as the final model and is evaluated on the test dataset to assess its performance.

3.5.2.3 Transformer-based Models

For this study, we used pre-trained *bert-base-uncased* model, the construction process involves several additional steps (see Listing 3.4 for psuedo-code):

- i. Load Tokenizer and Tokenize: Transformers require input data to be tokenized. This involves converting the input sequences into tokens that the model can process. So after the data preprocessing for transformers part, a pre-trained BERT tokenizer is loaded and each feature string is tokenized and padded to a maximum length of 128 tokens.
- ii. DataLoader Creation: In this step, a PyTorch `dataLoader` objects are created

```

1  Transformer-Based Model for Intrusion Detection
2  Inputs:  Dataset, RS1, RS2
3  Returns: Best Model Evaluation Metrics
4  Load dataset
5  # Preprocess dataset
6  if using RS1:
7      Convert timestamp for RS1
8  else if using RS2:
9      Remove timestamp for RS2
10 Convert arbitration_id and data_field to appropriate formats
11
12 # Extract features and labels
13 if using RS1:
14     X ← [datetime, arbitration_id, data_field]
15     y ← attack
16 else if using RS2:
17     X ← [arbitration_id, data_field]
18     y ← attack
19
20 # Scale features
21 Scale X using MinMaxScaler
22 # Prepare the final DataFrame
23 final_df ← DataFrame with features as feature_string and attack
24 # Tokenize the feature-strings
25 Initialize tokenizer from 'bert-base-uncased'
26 Tokenize sentences in final_df['features']
27 # Convert to tensors
28 Convert tokenized sentences and labels to tensors
29 # Split data into training, validation, and test sets
30 Split data into train, validation, and test sets
31 # Create DataLoader for training, validation, and test sets
32 Create DataLoader for train, validation, and test sets
33 # Load the pre-trained model
34 Load pre-trained BERT model for sequence classification
35 # Set up the optimizer and learning rate scheduler
36 Initialize optimizer and learning rate scheduler
37
38 # Training loop
39 for each epoch in epochs:
40     Train model on training data
41     Validate model on validation data
42     Save the best model based on validation loss with early
        stopping
43 # Load the best model
44 Load the best saved model
45 # Evaluate on the test set
46 Evaluate the best model on the test set
47 Calculate metrics(accuracy, precision, recall, F1 score, ROC AUC)
48 # Plot results
49 Plot confusion matrix, ROC curve, and metrics bar plot

```

Listing 3.4: Transformer-Based Model Pseudo-code

```

# Initialize the tokenizer
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')

# Tokenize the sentences
input_ids = []
attention_masks = []

max_length = 128 # Adjusted sequence length for better performance

for sent in sentences:
    encoded_dict = tokenizer.encode_plus(
        sent,
        add_special_tokens=True,
        max_length=max_length,
        pad_to_max_length=True,
        return_attention_mask=True,
        return_tensors='pt',
    )

    input_ids.append(encoded_dict['input_ids'])
    attention_masks.append(encoded_dict['attention_mask'])

input_ids = torch.cat(input_ids, dim=0)
attention_masks = torch.cat(attention_masks, dim=0)
labels = torch.tensor(labels)

```

Figure 3.5: Code snippet of BERT Tokenizer

```

# Load the pre-trained model
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2,
    output_attentions=False,
    output_hidden_states=False,
)

model.to(device)

```

Figure 3.6: Code snippet of BERT Model Loading

for the training, validation, and test sets to handle batching and shuffling of data during training and evaluation.

- iii. Load Pre-trained Model: Instead of training from scratch, a pre-trained BERT model *bert-base-uncased* is loaded using the *AutoModelForSequenceClassification* class from the transformers library. The model is configured for binary classification with two output labels.
- iv. Define Optimizer and Learning Rate Scheduler:
 - Optimization: In this step, the AdamW optimizer is used to adjust the model weights based on the computed loss.
 - Learning Rate Scheduler: In this step, a linear learning rate scheduler with warm-up is defined to adjust the learning rate during training, which helps in achieving better convergence..
- v. Model Training Loop: The training process involves iterating over the training dataset multiple times. In each iteration, the model makes predictions, computes the loss, and updates the weights using backpropagation.
- vi. Early Stopping Check: To prevent overfitting, early stopping is implemented. This monitors the model's performance on validation data and stops training if the performance does not improve after a certain number of epochs.
- vii. Load Best Model: The best model (with the lowest validation loss) is loaded for final evaluation.

3.5.3 Model Evalaution

In this research, we approached the problem as a binary classification task. This decision was informed by the observation that once an ECU is compromised, an attacker typically executes only one type of attack at any given time and there are other certain factors like ECUs computational power, network load, etc which do not allow to launch multiple attacks at same time.

Therefore, upon successfully training the models, we conducted a comprehensive evaluation using the testing data. For this assessment, we employed various evaluation

metrics widely recognized for their effectiveness in measuring the performance of classification models. These metrics include accuracy, precision, recall, F1 score, and the Receiver Operating Characteristic (ROC) Area Under the Curve (AUC) [60]. We also monitored the training and testing time required for the models to process 100k samples, providing insights into their computational efficiency. The details about metrics are explained below:

- **Accuracy:** This metric calculates the proportion of correctly predicted observations—both attacks and non-attacks—to the total samples in the dataset. It gives an overall indication of the model’s ability to accurately classify all scenarios.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- **Precision:** Defined as the ratio of correctly predicted attack instances to the total instances predicted as attacks. This metric is essential in contexts where falsely identifying normal operations as attacks (false positives) can be costly [61].

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** This metric measures the proportion of actual attacks that are correctly identified by the model, making it particularly critical where failing to detect an attack (false negatives) could lead to severe consequences [61].

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1 Score:** The F1 Score is a harmonic mean of Precision and Recall, balancing the trade-offs between them. This metric is especially useful when dealing with unbalanced datasets where attack samples may be less frequent [62].

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **ROC AUC:** The Receiver Operating Characteristic (ROC) curve illustrates

the model’s capability to discriminate between attack and non-attack instances across various thresholds. The Area Under the Curve (AUC) quantifies this ability, with higher values indicating superior model performance. The AUC is a measure of the likelihood that the model correctly ranks a randomly chosen attack instance more highly than a non-attack instance [63].

$$\text{TPR} = \frac{TP}{P} \quad \text{and} \quad \text{FPR} = \frac{FP}{N}$$

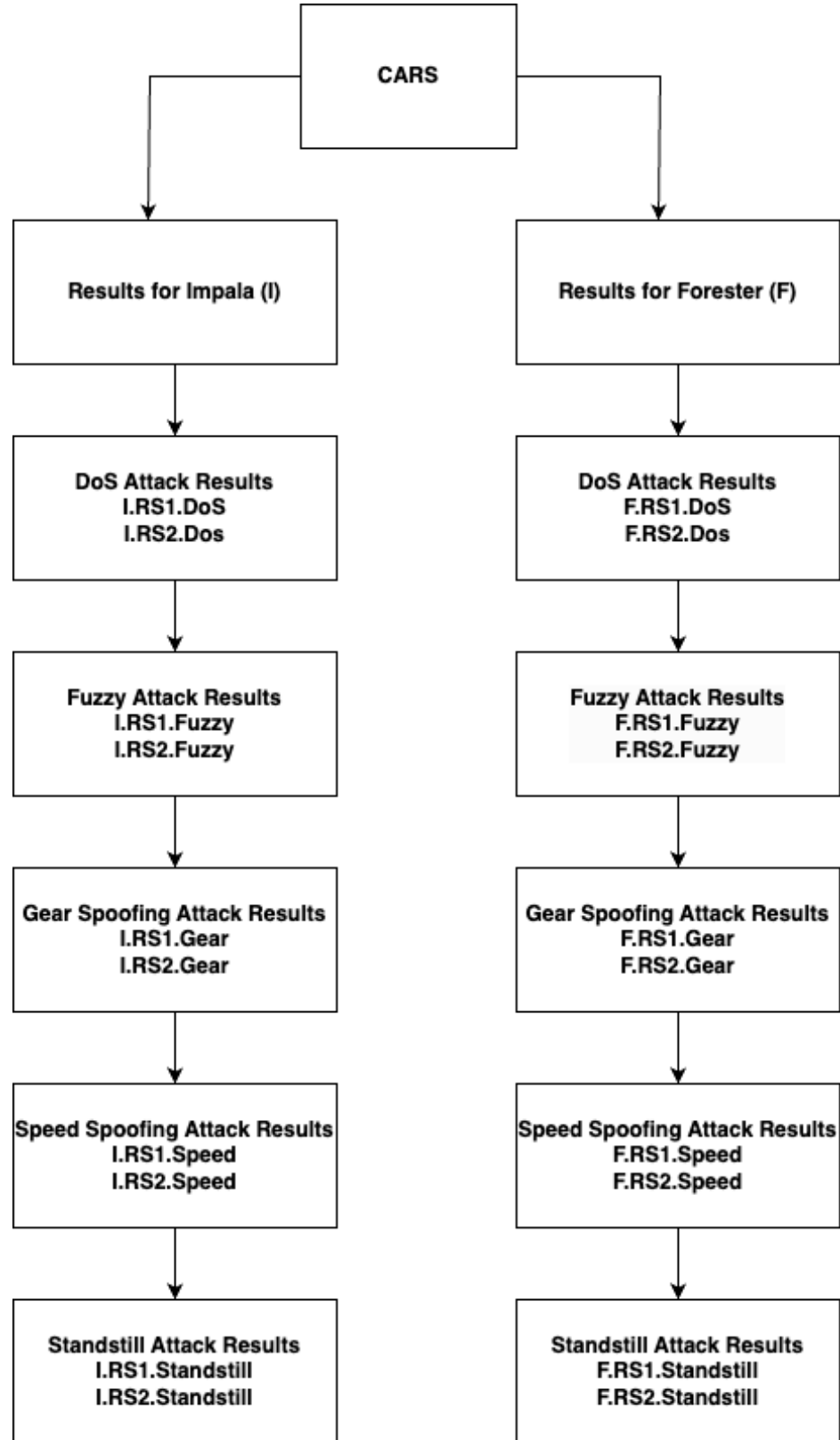


Figure 3.7: Representation of results for each attack type for both vehicles.

CHAPTER 4

Experiments and Results

In this chapter, we present a detailed analysis of the experiments conducted to evaluate the performance of the models used in HyCAN. The results are systematically categorized based on the type of vehicle and the specific attacks performed. This chapter aims to provide a comprehensive understanding of how the IDS performs under various attack scenarios, including DoS, Fuzzy, Gear, Speed, and Standstill attacks.

We begin by analyzing the results obtained from the experiments conducted on two vehicle models: the Impala and the Forester (see Fig. 3.7, explains flow of results for each attack type for both vehicles). Each subsection delves into the performance metrics, such as detection accuracy, false positive rate, and response time, for each type of attack on these vehicles. The experiments highlight the strengths of IDS, providing insights into its robustness and reliability in real-world scenarios.

Furthermore, this chapter differentiates the results of proposed approach from other existing approaches. Through this comparative analysis, we aim to validate the effectiveness of the IDS in providing enhanced security for in-vehicle networks against various attack vectors.

4.1 Results for Impala

This section represents the results for both research scenarios for all attacks performed on Impala vehicle.

4.1.1 DoS Attack

The results of the DoS attack detection on the Impala vehicle in both research scenarios reveal interesting trends across different models. Table 4.1 presents the results for Research Scenario 1, the Stacked model showed superior performance with the highest accuracy (96.69%) and vital metrics across Precision, Recall, and F1-score, indicating its high reliability for detecting DoS attacks. Models like CatBoost and XGBoost also performed well, though slightly lower, with CatBoost achieving 84.11% accuracy. BERT was noteworthy for its near-perfect ROC score (0.9220) but at the cost of significant computational time, with the highest training and prediction times across both scenarios. In Research scenario 2 (see Table 4.2), the Stacked model continued to perform strongly, though with a slight drop in accuracy (86.11%). In comparison, K-Nearest Neighbors outperformed other models in terms of Recall (90.58%) and achieved the highest accuracy (86.38%), indicating its particular effectiveness in this scenario. BERT again maintained high performance, but its computational costs were higher in RS2. Across both scenarios, Logistic Regression consistently showed the weakest performance, struggling with the lowest accuracy in both RS1 (51.36%) and RS2 (49.76%). Overall, while the Stacked model and BERT remained the most reliable across both scenarios, KNN demonstrated improved performance in RS2, suggesting variability in model effectiveness depending on the research scenario and attack conditions.

4.1.2 Fuzzy Attack

The results of the Fuzzing attack detection on the Impala vehicle in both research scenarios (RS1 and RS2) present several significant findings. In Research Scenario 1 (see Table 4.3), the Stacked model showed superior performance with the highest accuracy (97.01%) and consistently high scores across Precision (96.93%), Recall (97.05%), and F1-score (96.99%). This makes it the most reliable model for Fuzzing attack detection in this scenario. BERT also performed exceptionally well, with an accuracy of 93.30% and a strong ROC score (0.9302), though, as in the previous attack type,

Table 4.1: IRS1.DoS - Impala results for Research Scenario 1 for DoS attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5136	0.5119	0.7096	0.5948	0.4988	0.60	0.10
KNN	0.5578	0.5622	0.5471	0.5545	0.7083	8.71	0.86
DT	0.7720	0.8440	0.6707	0.7425	0.8928	4.00	0.01
RF	0.7611	0.8008	0.6991	0.7465	0.8841	310.61	0.08
XGBoost	0.8268	0.8798	0.7595	0.8152	0.9328	16.31	0.10
Cat Boost	0.8411	0.8543	0.8247	0.8392	0.9494	279.47	0.01
Stacked	0.9669	0.9669	0.9673	0.9671	0.9941	49.35	0.42
MLP	0.5583	0.5657	0.5547	0.5598	0.6113	105.24	0.50
LSTM	0.8763	0.8762	0.8778	0.8778	0.8800	72.06	0.41
BERT	0.9231	0.9250	0.9205	0.9198	0.9220	4260.79	39.50

Table 4.2: IRS2.DoS - Impala results for Research Scenario 2 for DoS attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.4976	0.4978	0.7318	0.5925	0.4960	0.59	0.01
KNN	0.8638	0.8352	0.9058	0.8691	0.9584	11.94	0.21
DT	0.7670	0.7922	0.7227	0.7559	0.8226	0.77	0.02
RF	0.7692	0.7659	0.7920	0.7740	0.8907	111.90	0.02
XGBoost	0.8237	0.8846	0.7438	0.8081	0.9338	15.50	0.01
Cat Boost	0.8331	0.9096	0.7390	0.8155	0.9422	281.91	0.01
Stacked	0.8611	0.8378	0.8950	0.8655	0.9591	28.93	0.43
MLP	0.5657	0.5601	0.6379	0.5965	0.6093	130.15	0.25
LSTM	0.5044	0.4981	0.4996	0.3792	0.5000	25.89	0.57
BERT	0.8612	0.8618	0.8598	0.8598	0.8600	4146.08	67.07

BERT required a longer training time (2817.93 seconds) and prediction time (28.34 seconds). Models such as XGBoost and CatBoost maintained strong performance, with XGBoost achieving 82.54% accuracy and a high F1-score of 81.89%. In contrast, Logistic Regression and MLP showed lower performance, with LR achieving an accuracy of 49.36%, indicating these models struggle with Fuzzing attack detection in RS1. In Research Scenario 2 (see Table 4.4), the Stacked model remained a top performer, though with a slight drop in accuracy to 86.47%. KNN, however, showed significant improvement in RS2, achieving the highest accuracy (86.77%) and Recall (88.89%), making it particularly effective in this scenario. BERT also continued to perform well in RS2 with an accuracy of 86.22%, though its computational demands increased with a training time of 3687.31 seconds and prediction time of 66.15 seconds. Once again, Logistic Regression and MLP struggled, with LR displaying the weakest performance in RS2 with an accuracy of 49.11%. Overall, the Stacked model and BERT maintained strong performance across both scenarios, though KNN emerged as a highly effective model in RS2, reflecting the variability in model performance across different attack scenarios.

4.1.3 Gear Spoofing Attack

The results for detecting Gear Spoofing attacks on the Impala vehicle show variability across models. Table 4.5 presents the results for Research Scenario 1, the Stacked model outperformed others with the highest accuracy of 94.48%, alongside strong Precision (94.42%), Recall (94.56%), and F1-score (94.49%), making it the most reliable model for Gear Spoofing attack detection in this scenario. BERT also performed well with an accuracy of 86.69% and a ROC of 0.8700. However, like previous scenarios, it incurred a high computational cost with a training time of 5943.44 seconds and a prediction time of 68.67 seconds. XGBoost and CatBoost performed decently, with accuracies of 73.39% and 74.51%, respectively, and CatBoost showing a higher F1-score of 73.36%. On the other hand, Logistic Regression and KNN struggled, with LR achieving an accuracy of only 51.46% and KNN 44.56%, indicating their lower suitability for detecting Gear Spoofing attacks in RS1. In Research Scenario 2 (see

Table 4.3: I.RS1.Fuzzy - Impala results for Research Scenario 1 for Fuzzy attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.4936	0.4916	0.2831	0.3593	0.4932	0.50	0.10
KNN	0.5421	0.5436	0.5424	0.5430	0.6822	7.86	0.69
DT	0.7770	0.7302	0.8801	0.7985	0.8997	4.43	0.20
RF	0.7597	0.7601	0.7544	0.7573	0.8883	300.59	0.08
XGBoost	0.8254	0.8449	0.7943	0.8189	0.9387	16.05	0.01
Cat Boost	0.8435	0.8511	0.8303	0.8405	0.9517	285.32	0.01
Stacked	0.9701	0.9693	0.9705	0.9699	0.9949	52.32	0.43
MLP	0.5031	0.5031	0.4501	0.6694	0.5001	29.29	0.25
LSTM	0.8763	0.8762	0.8778	0.8778	0.8800	72.06	0.41
BERT	0.9330	0.9374	0.9330	0.9328	0.9300	2817.93	28.34

Table 4.4: I.RS2.Fuzzy - Impala results for Research Scenario 2 for Fuzzy attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.4911	0.4831	0.0253	0.0481	0.5008	0.54	0.03
KNN	0.8677	0.8562	0.8889	0.8722	0.9260	9.09	0.17
DT	0.7730	0.7762	0.7774	0.7768	0.8980	0.84	0.04
RF	0.7746	0.7969	0.7546	0.7709	0.8987	99.89	0.02
XGBoost	0.8194	0.8778	0.7586	0.8081	0.9328	15.32	0.01
Cat Boost	0.8344	0.8778	0.7831	0.8277	0.9452	254.95	0.05
Stacked	0.8647	0.8473	0.8948	0.8704	0.9619	25.42	0.39
MLP	0.5598	0.5487	0.6702	0.6034	0.6000	134.36	0.25
LSTM	0.5169	0.5191	0.5154	0.4913	0.5000	50.55	0.53
BERT	0.8622	0.8854	0.8627	0.8602	0.8600	3687.31	66.15

Table 4.6), the Stacked model again led with a high accuracy of 86.74%, although it dropped compared to RS1. BERT continued to show strong performance with an accuracy of 77.49% and the highest F1-score (77.32%) in RS2. KNN showed a notable improvement in RS2, achieving an accuracy of 76.93% and the highest Recall (77.89%), demonstrating better detection of Gear Spoofing attacks in this scenario. XGBoost and CatBoost maintained respectable performance, with XGBoost achieving an accuracy of 72.36%. Meanwhile, Logistic Regression and MLP remained the weakest performers in RS2, with LR showing an accuracy of 52.07% and MLP at 54.06%. Overall, while the Stacked model and BERT consistently performed well across both scenarios, KNN showed substantial improvement in RS2, reinforcing the variability in model effectiveness depending on the attack scenario and conditions.

Table 4.5: I.RS1.Gear - Impala results for Research Scenario 1 for Gear Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5146	0.5278	0.3018	0.3840	0.5337	0.57	0.10
KNN	0.4456	0.4481	0.4498	0.4490	0.5492	8.39	0.71
DT	0.6892	0.6908	0.6881	0.6894	0.8056	5.08	0.20
RF	0.6718	0.7162	0.5720	0.6360	0.7969	351.73	0.32
XGBoost	0.7339	0.7516	0.7009	0.7254	0.8605	16.49	0.01
Cat Boost	0.7451	0.7704	0.7003	0.7336	0.8754	294.00	0.01
Stacked	0.9448	0.9442	0.9456	0.9449	0.9864	51.03	0.44
MLP	0.5314	0.5451	0.3520	0.4277	0.5515	116.56	0.24
LSTM	0.8263	0.8362	0.8220	0.8220	0.8300	67.06	0.41
BERT	0.8669	0.8769	0.8667	0.8660	0.8700	5943.44	68.67

Table 4.6: I.RS2.Gear - Impala results for Research Scenario 2 for Gear Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5207	0.5390	0.3075	0.3916	0.5336	0.48	0.01
KNN	0.7693	0.7654	0.7789	0.7721	0.8850	9.32	0.21
DT	0.6907	0.6921	0.6906	0.6913	0.8069	0.71	0.06
RF	0.6941	0.7771	0.5470	0.6421	0.8072	95.27	0.02
XGBoost	0.7236	0.8094	0.5874	0.6807	0.8436	14.23	0.01
Cat Boost	0.7395	0.8041	0.6536	0.7100	0.8590	264.09	0.01
Stacked	0.7685	0.7898	0.7337	0.7607	0.8802	24.79	0.43
MLP	0.5406	0.5795	0.2700	0.3683	0.5549	119.98	0.25
LSTM	0.5191	0.5206	0.5175	0.4994	0.5200	51.29	0.58
BERT	0.7749	0.8270	0.7732	0.7648	0.7700	2491.83	28.11

4.1.4 Speed Spoofing Attack

The Speed Spoofing attack detection results on the Impala vehicle reveal interesting patterns across models in both RS1 and RS2. In Research Scenario 1 (see Table 4.7), the Stacked model performed the best with an outstanding accuracy of 96.65%, along with high Precision (96.77%), Recall (96.57%), and F1-score (96.67%), making it the most effective model for detecting Speed Spoofing attacks in this scenario. BERT also demonstrated excellent performance with an accuracy of 92.69% and a ROC score of 0.9330, but like in previous scenarios, it required the most computational resources, with a training time of 4846.51 seconds and a prediction time of 70.57 seconds. XGBoost and CatBoost were also strong contenders, achieving accuracies of 82.29% and 83.33%, respectively, with CatBoost showing a slightly higher F1-score of 83.33%. On the lower end, Logistic Regression and KNN struggled, with LR displaying the weakest performance in RS1, with an accuracy of 50.11%, and KNN achieving 55.82%. Table 4.8 presents results for Research Scenario 2, the Stacked model remained a top performer with an accuracy of 85.00%, though it saw a slight decline compared to RS1. KNN showed significant improvement in RS2, achieving the highest accuracy (85.10%) and the best Recall (88.35%), indicating its enhanced capability to detect true positives in this scenario. BERT again performed well in RS2 with an accuracy of 85.31%. However, it continued to be computationally expensive, with a training time of 2965.62 seconds and a prediction time of 68.51 seconds. XGBoost and CatBoost maintained competitive performance in RS2, with XGBoost achieving an accuracy of 81.36% and CatBoost at 82.00%. As in previous cases, Logistic Regression and MLP were the weakest performers, with LR reaching an accuracy of 49.74% and MLP at 57.05%. Overall, both scenarios showed that the Stacked model and BERT remained strong. In contrast, KNN showed substantial improvement in RS2, indicating that the detection capability of models varies depending on the attack scenario.

Table 4.7: I.RS1.Speed - Impala results for Research Scenario 1 for Speed Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5011	0.5123	0.2392	0.3261	0.5183	0.56	0.10
KNN	0.5582	0.5629	0.5572	0.5600	0.6976	8.52	0.79
DT	0.7735	0.7667	0.7923	0.7792	0.8961	4.52	0.29
RF	0.7447	0.7724	0.7004	0.7347	0.8707	296.06	0.08
XGBoost	0.8229	0.8491	0.7894	0.8182	0.9362	15.56	0.19
Cat Boost	0.8333	0.8413	0.8254	0.8333	0.9452	275.23	0.01
Stacked	0.9665	0.9677	0.9657	0.9667	0.9933	48.43	0.39
MLP	0.5601	0.5645	0.5207	0.5417	0.6024	126.29	0.24
LSTM	0.8763	0.8762	0.8778	0.8778	0.8800	72.06	0.41
BERT	0.9269	0.9329	0.9294	0.9292	0.9330	4846.51	70.57

Table 4.8: I.RS2.Speed - Impala results for Research Scenario 2 for Speed Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.4974	0.4937	0.2727	0.3513	0.5105	0.72	0.02
KNN	0.8510	0.8291	0.8835	0.8554	0.9523	13.02	0.25
DT	0.7710	0.7815	0.7513	0.7661	0.8959	1.08	0.06
RF	0.7727	0.7455	0.8268	0.7840	0.8968	150.06	0.02
XGBoost	0.8136	0.8529	0.7570	0.8021	0.9260	21.63	0.01
Cat Boost	0.8202	0.8155	0.8269	0.8211	0.9357	403.54	0.01
Stacked	0.8500	0.8502	0.8489	0.8496	0.9525	31.31	0.52
MLP	0.5705	0.5749	0.5354	0.5544	0.6169	83.34	0.26
LSTM	0.5053	0.5058	0.5041	0.4671	0.5000	36.48	0.60
BERT	0.8531	0.8574	0.8533	0.8527	0.8500	2965.62	68.51

4.1.5 Standstill Attack

The detection results for the Standstill attack on the Impala vehicle exhibit notable differences in model performance in both research scenarios. In Research Scenario 1 (see Table 4.9), the Stacked model significantly outperformed all other models with very high accuracy of 90.03%, along with strong Precision (90.06%), Recall (90.11%), and F1-score (90.08%). This makes the Stacked model the most effective in this scenario. LSTM also performed well, with an accuracy of 67.63% and an F1-score of 67.78%, indicating it is a viable alternative, though still significantly behind the Stacked model. BERT demonstrated a moderate performance in RS1 with an accuracy of 54.07%, though its computational time was considerably high, with a training time of 2371.03 seconds and a prediction time of 68.70 seconds. XGBoost and CatBoost did not perform well in RS1, achieving accuracies of 48.63% and 48.76%, respectively. At the lower end, KNN struggled the most with an accuracy of only 26.89%. In Research Scenario 2 (see Table 4.10), the Stacked model again led with an accuracy of 54.57%, though it dropped significantly compared to RS1. LSTM showed slight improvement in RS2, reaching an accuracy of 49.18%, while BERT maintained a similar performance, with an accuracy of 49.90%. KNN improved in RS2, achieving an accuracy of 47.32%, though its Recall (51.22%) was better than most models. Logistic Regression consistently showed the weakest performance in both scenarios, with an accuracy of 49.13% in RS1 and 49.80% in RS2. Across both scenarios, the Stacked model remained the top performer, though its advantage was more pronounced in RS1. Most models saw a decline in RS2, highlighting the increased difficulty in detecting Standstill attacks in this scenario.

4.2 Results for Forester

This section represents the results for both research scenarios for all attacks performed on Forester vehicle.

Table 4.9: I.RS1.Standstill - Impala results for Research Scenario 1 for Standstill attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.4913	0.4920	0.3779	0.4275	0.4912	0.53	0.10
KNN	0.2689	0.2698	0.2666	0.2682	0.2368	8.92	0.76
DT	0.4800	0.4847	0.5532	0.5167	0.4697	4.10	0.10
RF	0.4369	0.4453	0.4914	0.4672	0.3977	377.37	0.02
XGBoost	0.4863	0.4869	0.4169	0.4492	0.4792	15.09	0.10
Cat Boost	0.4876	0.3096	0.3778	0.4763	0.4800	245.44	0.01
Stacked	0.9003	0.9006	0.9011	0.9008	0.9800	54.28	0.45
MLP	0.5011	0.2933	0.2900	0.2700	0.5000	44.35	0.27
LSTM	0.6763	0.6762	0.6778	0.6778	0.6100	52.06	0.41
BERT	0.5407	0.2524	0.5047	0.3386	0.5000	2371.03	68.70

Table 4.10: I.RS2.Standstill - Impala results for Research Scenario 2 for Standstill attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.4980	0.2000	0.2000	0.2000	0.4980	0.53	0.10
KNN	0.4732	0.4770	0.5122	0.4939	0.4695	10.95	0.90
DT	0.4785	0.4773	0.4091	0.4405	0.4658	0.76	0.05
RF	0.4780	0.4814	0.5155	0.4979	0.4642	98.02	0.02
XGBoost	0.4929	0.4610	0.0607	0.1072	0.4839	13.33	0.04
Cat Boost	0.4904	0.4902	0.3819	0.4293	0.4808	218.16	0.09
Stacked	0.5457	0.5555	0.4755	0.5124	0.5745	23.81	0.49
MLP	0.5024	0.2300	0.2340	0.2500	0.5000	18.79	0.28
LSTM	0.4918	0.4877	0.4970	0.3778	0.5000	31.01	0.53
BERT	0.4990	0.2495	0.5000	0.3329	0.5000	4696.33	66.11

4.2.1 DoS Attack

The detection results for the DoS attack on the Forester vehicle show apparent variations in model performance between RS1 and RS2, mainly due to the exclusion of the timestamp feature in RS2. In Research Scenario 1 (see Table 4.11), the BERT model significantly outperformed others, achieving near-perfect accuracy (99.53%) along with strong Precision (99.52%), Recall (99.53%), and F1-score (99.52%). However, it required a high computational cost with a training time of 5929.72 seconds. The Stacked model also performed well in RS1 with an accuracy of 91.23% and an F1-score of 91.20%, followed by LSTM with 87.63% accuracy, making these two models strong alternatives to BERT. Models like XGBoost and CatBoost demonstrated robust performance, with accuracies of 80.38% and 80.75%, respectively, while Logistic Regression and KNN struggled, achieving 53.29% and 60.29% accuracy, respectively. In Research Scenario 2 (see Table 4.12), removing the timestamp feature led to noticeable shifts in performance. KNN saw a remarkable improvement, achieving the highest accuracy in RS2 (80.92%) along with strong Precision and Recall. Both the Stacked model and BERT experienced declines in RS2, with the Stacked model dropping to 81.12% accuracy and BERT to 79.84%, a significant reduction from their RS1 results. While XGBoost and CatBoost maintained competitive performance in RS2 with accuracies of 75.92% and 76.37%, LSTM faced a sharp drop in accuracy, falling to 53.34%, indicating its reliance on temporal data. Logistic Regression showed consistently poor performance across both scenarios, with minimal change in accuracy between RS1 and RS2. Overall, BERT and the Stacked model were the top performers in RS1. At the same time, KNN improved significantly in RS2, underscoring the impact of the timestamp feature in enhancing model performance, particularly for temporal-sensitive models like LSTM and BERT.

4.2.2 Fuzzy Attack

The detection results for the Fuzzing attack on the Forester vehicle show noticeable differences. Table 4.13 presents results for Research Scenario 1, the BERT model out-

Table 4.11: F.RS1.DoS - Forester results for Research Scenario 1 for DoS attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5329	0.5539	0.3233	0.4083	0.5166	0.64	0.01
KNN	0.6029	0.6012	0.6041	0.6026	0.7592	10.00	0.70
DT	0.7462	0.7424	0.7515	0.7569	0.8509	4.13	0.01
RF	0.7358	0.7552	0.6953	0.7240	0.8584	361.83	0.37
XGBoost	0.8038	0.7964	0.8145	0.8054	0.9229	19.40	0.01
Cat Boost	0.8075	0.7960	0.8253	0.8104	0.9263	322.87	0.01
Stacked	0.9123	0.9120	0.9119	0.9120	0.9731	54.36	0.46
MLP	0.6419	0.6181	0.7306	0.6696	0.7305	148.27	0.27
LSTM	0.8763	0.8762	0.8778	0.8778	0.8800	72.06	0.41
BERT	0.9953	0.9952	0.9953	0.9952	1.000	5929.72	65.42

Table 4.12: F.RS2.DoS - Forester results for Research Scenario 2 for DoS attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5283	0.5414	0.3194	0.4018	0.5159	0.61	0.01
KNN	0.8092	0.8066	0.8092	0.8079	0.9091	8.61	0.18
DT	0.7295	0.7356	0.7093	0.7222	0.8456	1.11	0.01
RF	0.7307	0.7332	0.7183	0.7257	0.8534	158.27	0.24
XGBoost	0.7592	0.7744	0.7260	0.7494	0.8791	15.66	0.01
Cat Boost	0.7637	0.7856	0.7198	0.7513	0.8830	314.69	0.01
Stacked	0.8112	0.8039	0.8189	0.8113	0.9204	33.90	0.46
MLP	0.5776	0.7445	0.2500	0.3743	0.6309	208.79	0.44
LSTM	0.5334	0.4991	0.5996	0.4792	0.5000	32.89	0.57
BERT	0.7984	0.8121	0.7985	0.7962	0.8000	1563.78	28.23

performed others with near-perfect accuracy (99.16%) and strong Precision, Recall, and F1-score. However, it incurred a high computational cost, with the longest training time of 3506.71 seconds. The Stacked model also performed well, achieving 90.49% accuracy, followed by LSTM with 85.53% making them reliable alternatives. XGBoost and CatBoost delivered strong results with accuracies of 80.43% and 80.24%, respectively, while KNN and Logistic Regression struggled, with accuracies of 60.14% and 52.38%. In Research Scenario 2 (see Table 4.14), the exclusion of the timestamp feature caused significant changes in performance. KNN showed the most improvement, achieving the highest accuracy in RS2 (81.40%) along with strong Recall (79.72%). Meanwhile, both BERT and the Stacked model saw reduced performance, with BERT falling to 80.21% and the Stacked model to 81.02% , though they remained strong overall. XGBoost and CatBoost also experienced a decline, with accuracies of 75.54% and 76.23%. LSTM saw a sharp drop, falling to 51.69%, indicating its reliance on the temporal information provided by the timestamp feature. Overall, while BERT and the Stacked model were top performers in RS1, KNN emerged as the best model in RS2, highlighting the impact of excluding the timestamp feature, which affected the performance of temporal-sensitive models like LSTM and BERT.

4.2.3 Gear Spoofing Attack

The detection results for the Gear Spoofing attack on the Forester vehicle show distinct performance. In Research Scenario 1 (see Table 4.15), the BERT model outperformed others with near-perfect accuracy (99.91%), along with strong Precision, Recall, and F1-score (99.91%), although it required the highest computational resources with a training time of 4131.46 seconds. The Stacked model also performed well with 91.85% accuracy, followed by XGBoost and CatBoost, which achieved 82.41% and 82.83% accuracy, respectively. LSTM was another strong performer with 82.63% accuracy, while models like Logistic Regression and KNN lagged behind, with accuracies of 51.67% and 62.40%, respectively. In Research Scenario 2 (see Table 4.16), removing the timestamp feature led to significant changes in performance. Logistic Regression improved dramatically, achieving the highest accuracy in RS2 at 87.66%,

Table 4.13: F.RS2.Fuzzy - Forester results for Research Scenario 1 for Fuzzy attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5238	0.5425	0.3129	0.3969	0.5218	0.56	0.01
KNN	0.6014	0.6020	0.6016	0.6018	0.7592	9.07	0.81
DT	0.7422	0.7096	0.8212	0.7613	0.8592	3.48	0.05
RF	0.7319	0.7376	0.7212	0.7293	0.8572	302.57	0.17
XGBoost	0.8043	0.8132	0.7908	0.8019	0.9250	16.45	0.01
Cat Boost	0.8024	0.7978	0.8109	0.8043	0.9238	279.78	0.01
Stacked	0.9049	0.9093	0.8998	0.9045	0.9704	49.89	0.43
MLP	0.6419	0.6181	0.7306	0.6696	0.7305	28.28	0.25
LSTM	0.8553	0.8762	0.8698	0.8678	0.8800	75.06	1.09
BERT	0.9916	0.9916	0.9816	0.9916	1.000	3506.71	63.85

Table 4.14: F.RS2.Fuzzy - Forester results for Research Scenario 2 for Fuzzy attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5155	0.5426	0.2999	0.3863	0.5157	0.60	0.02
KNN	0.8140	0.8302	0.7972	0.8134	0.9096	9.71	0.16
DT	0.7270	0.7717	0.6576	0.7101	0.8455	1.19	0.02
RF	0.7297	0.7380	0.7261	0.7320	0.8518	142.75	0.23
XGBoost	0.7554	0.7900	0.7067	0.7460	0.8769	14.51	0.03
Cat Boost	0.7623	0.8094	0.6965	0.7487	0.8812	263.20	0.02
Stacked	0.8102	0.8208	0.8215	0.8110	0.9194	30.52	0.39
MLP	0.5726	0.6345	0.3640	0.4626	0.6228	203.09	0.43
LSTM	0.5169	0.5191	0.5154	0.4913	0.5000	50.55	0.53
BERT	0.8021	0.8172	0.8023	0.7998	0.8000	1877.20	28.25

while BERT saw a noticeable decline in performance, dropping to 88.23% accuracy, though it remained among the top models. The Stacked model and CatBoost continued to deliver competitive results in RS2, with accuracies of 87.69% and 81.63%, respectively. Meanwhile, XGBoost and Random Forest maintained strong performance in RS2, each with around 81% accuracy. However, LSTM, which performed well in RS1, experienced a sharp decline, dropping to 51.91% accuracy, indicating its reliance on the timestamp feature. Overall, while BERT and the Stacked model were the strongest performers in RS1, Logistic Regression showed the most improvement in RS2, underscoring the impact of temporal data on model performance.

Table 4.15: F.RS1.Gear - Forester results for Research Scenario 1 for Gear Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5167	0.5345	0.2933	0.3787	0.5574	0.56	0.01
KNN	0.6240	0.6275	0.6184	0.6229	0.7795	8.19	0.70
DT	0.7934	0.8182	0.7567	0.7863	0.8949	4.52	0.02
RF	0.7300	0.7274	0.7396	0.7335	0.8651	325.25	0.02
XGBoost	0.8241	0.8397	0.8030	0.8209	0.9381	14.98	0.01
Cat Boost	0.8283	0.8451	0.8059	0.8250	0.9428	273.85	0.01
Stacked	0.9185	0.9180	0.9199	0.9190	0.9722	53.96	0.39
MLP	0.6310	0.6233	0.6459	0.6344	0.7206	146.56	0.27
LSTM	0.8263	0.8362	0.8220	0.8220	0.8300	67.06	0.41
BERT	0.9991	0.9991	0.9998	0.9991	1.000	4131.46	66.32

4.2.4 Speed Spoofing Attack

The detection results for the Speed Spoofing attack on the Forester vehicle show noticeable differences between RS1 and RS2. In Research Scenario 1 (see Table 4.17),

Table 4.16: F.RS2.Gear - Forester results for Research Scenario 2 for Gear Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.8766	0.8705	0.8834	0.8774	0.9354	13.35	0.17
KNN	0.5170	0.5296	0.2907	0.3754	0.5564	0.79	0.02
DT	0.7910	0.7594	0.8510	0.8026	0.9105	1.94	0.04
RF	0.7918	0.7590	0.8541	0.8093	0.9113	227.73	0.03
XGBoost	0.8119	0.7778	0.8723	0.8224	0.9285	21.01	0.01
Cat Boost	0.8163	0.7830	0.8743	0.8262	0.9317	401.83	0.01
Stacked	0.8769	0.8725	0.8823	0.8774	0.9650	40.89	0.49
MLP	0.6250	0.5968	0.8025	0.6845	0.6959	209.67	0.43
LSTM	0.5191	0.5206	0.5175	0.4994	0.5200	51.29	0.58
BERT	0.8823	0.8866	0.8826	0.8820	0.8800	1562.27	28.22

the BERT model significantly outperformed all other models, achieving near-perfect accuracy (99.47%) along with strong Precision, Recall, and F1-score (99.47%). However, it required the highest computational cost, with a training time of 5936.78 seconds. The Stacked model also performed well in RS1 with an accuracy of 91.53% and an F1-score of 91.96%, followed by CatBoost (83.10%) and XGBoost (81.86%), both of which demonstrated strong overall performance. LSTM also showed decent results with 82.63% accuracy, while models like Logistic Regression and KNN struggled, achieving lower accuracies of 50.26% and 64.57%, respectively. In Research Scenario 2 (see Table 4.18), excluding the timestamp feature caused notable changes in model performance. KNN improved significantly, achieving the highest accuracy in RS2 at 86.99%, with strong Precision (87.67%) and Recall (85.90%). Meanwhile, BERT saw a considerable drop in accuracy, falling to 87.13%, though it remained a top performer. The Stacked model and CatBoost continued to deliver competitive results in RS2, with accuracies of 86.53% and 81.22%, respectively. Other models such as XGBoost and Random Forest maintained strong performances, with accuracies around 81% and 78%, respectively. However, LSTM experienced a significant decline in RS2, with its accuracy dropping to 49.53%, indicating its dependence on the timestamp feature. Overall, while BERT and the Stacked model led in RS1, KNN showed substantial improvement in RS2, emphasizing the importance of temporal data for models like LSTM and BERT.

4.2.5 Standstill Attack

The detection results for the Standstill attack on the Forester vehicle show clear performance differences between RS1 and RS2. In Research Scenario 1 (see Table 4.19), the BERT model dominated with a near-perfect accuracy of 99.90%, along with strong Precision, Recall, and F1-score (99.99%), though it required significant computational resources, with a training time of 5889.03 seconds. The Stacked model also performed exceptionally well, with an accuracy of 92.38% and an F1-score of 92.44%, followed by CatBoost and XGBoost, which achieved accuracies of 83.84% and 82.94%, respectively. LSTM also delivered solid results with 79.63% accuracy, while

Table 4.17: F.RS1.Speed - Forester results for Research Scenario 1 for Speed Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5026	0.5104	0.2703	0.3534	0.5708	0.57	0.01
KNN	0.6457	0.6501	0.6402	0.6451	0.7870	9.35	0.72
DT	0.7932	0.8257	0.7463	0.7840	0.8903	3.96	0.02
RF	0.7557	0.8186	0.6607	0.7312	0.8750	320.53	0.17
XGBoost	0.8186	0.8224	0.8154	0.8189	0.9357	16.95	0.01
Cat Boost	0.8310	0.8313	0.8331	0.8322	0.9421	285.15	0.01
Stacked	0.9153	0.9125	0.9197	0.9196	0.9708	55.61	0.43
MLP	0.6688	0.6630	0.6835	0.6731	0.7543	147.30	0.26
LSTM	0.8263	0.8362	0.8478	0.8278	0.8500	68.98	1.25
BERT	0.9947	0.9947	0.9947	0.9947	1.000	5936.78	68.15

Table 4.18: F.RS2.Speed - Forester results for Research Scenario 2 for Speed Spoofing attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5126	0.5178	0.2816	0.3648	0.5708	0.72	0.02
KNN	0.8699	0.8767	0.8590	0.8677	0.9289	11.67	0.17
DT	0.7814	0.7463	0.8487	0.7942	0.9006	1.38	0.05
RF	0.7837	0.7548	0.8364	0.7935	0.9056	212.74	0.14
XGBoost	0.8127	0.8044	0.8235	0.8138	0.9254	23.74	0.01
Cat Boost	0.8122	0.7930	0.8420	0.8167	0.9281	394.01	0.01
Stacked	0.8653	0.8661	0.8623	0.8642	0.9579	39.68	0.47
MLP	0.6307	0.7623	0.3865	0.5129	0.7321	204.57	0.43
LSTM	0.4953	0.5558	0.5241	0.4671	0.5000	56.58	0.60
BERT	0.8713	0.8832	0.8732	0.8731	0.8700	4130.48	66.95

models like KNN and Logistic Regression struggled, achieving 62.25% and 49.88%, respectively. In Research Scenario 2 (see Table 4.20), removing the timestamp feature led to significant shifts in model performance. KNN showed remarkable improvement, achieving the highest accuracy in RS2 at 86.23%, with strong Precision (85.27%) and F1-score (86.48%). Meanwhile, BERT saw a substantial decline, dropping to 88.99% accuracy, although it remained a top.

Table 4.19: F.RS1.Standstill - Forester results for Research Scenario 1 for Standstill attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.4988	0.4992	0.2654	0.3456	0.5214	0.55	0.09
KNN	0.6225	0.6256	0.6133	0.6194	0.7752	9.24	0.67
DT	0.7789	0.8402	0.6897	0.7575	0.8712	3.94	0.02
RF	0.7453	0.7409	0.7555	0.7481	0.8597	315.41	0.33
XGBoost	0.8294	0.8306	0.8282	0.8294	0.9440	16.38	0.01
Cat Boost	0.8384	0.8467	0.8270	0.8276	0.9508	278.89	0.01
Stacked	0.9238	0.9192	0.9296	0.9244	0.9734	50.19	0.41
MLP	0.6374	0.6072	0.7952	0.6886	0.7270	142.04	0.26
LSTM	0.7963	0.7862	0.7778	0.7978	0.8000	52.06	0.41
BERT	0.9990	0.9997	0.9899	0.9990	1.000	5889.03	65.32

4.3 Results Discussion

The analysis of various attacks on the Impala and Forester vehicles across different research scenarios (RS1 and RS2) highlights notable differences in model performance, largely driven by the inclusion or exclusion of the timestamp feature. In Research Scenario 1, models like the BERT and Stacked model consistently performed exceptionally well, especially for timing-sensitive attacks like DoS, Gear Spoofing, and

Table 4.20: F.RS2.Standstill - Forester results for Research Scenario 2 for Standstill attack

MODEL	Accuracy	Precision	Recall	F1	ROC	TT	PT
LR	0.5029	0.5093	0.2667	0.3501	0.5203	0.61	0.02
KNN	0.8623	0.8527	0.8772	0.8648	0.9349	8.95	0.16
DT	0.7722	0.7824	0.7565	0.7693	0.8923	1.30	0.03
RF	0.7743	0.7750	0.7756	0.7753	0.8962	152.03	0.11
XGBoost	0.8063	0.7894	0.8377	0.8128	0.9239	14.95	0.01
Cat Boost	0.8121	0.8081	0.8205	0.8143	0.9298	281.44	0.01
Stacked	0.8619	0.8467	0.8838	0.8653	0.9556	32.34	0.39
MLP	0.6067	0.7936	0.3047	0.4404	0.6864	173.25	0.43
LSTM	0.4418	0.4977	0.5070	0.3978	0.5000	31.01	0.53
BERT	0.8899	0.8981	0.8896	0.8892	0.8900	3530.69	67.21

Standstill, achieving near-perfect accuracy, Precision, Recall, and F1-scores. However, in RS2, where the timestamp feature was excluded, the performance of models like BERT and the Stacked model declined noticeably across all metrics, underscoring the critical role of the timestamp in attack detection. KNN showed improvement in RS2 for specific attacks, particularly Speed Spoofing and Standstill, but still lagged behind BERT and the Stacked model. XGBoost, CatBoost, and LSTM maintained decent performance in both scenarios but were impacted by the absence of temporal data in Research Scenario 2.

The Stacked model is the most suitable option for real-world deployment, balancing high accuracy with reasonable computational efficiency. While BERT performs exceptionally well, its computational demands may be impractical for real-time environments. The framework is robust for detecting real-world attacks, particularly with the timestamp feature, as it helps capture temporal patterns essential for detecting timing-based attacks. To ensure reliable and accurate detection of various in-vehicle attacks, preserving temporal relationships in the data, such as the timestamp, is critical.

4.4 Difference from other existing approaches

In this research, we introduce HyCAN, a novel framework designed to enhance in-vehicle network security by integrating traditional machine learning algorithms and advanced deep learning architectures, including Transformer-based models. The primary purpose of this study is to cover the research gaps in existing studies that only focus on working on a single type of approach. Our hybrid approach stands out in automotive cybersecurity by offering a versatile and comprehensive solution that achieves high detection accuracy and addresses the practical challenges of computational efficiency and real-world applicability.

As the direct comparison with other approaches is not possible because each approach focused on single type of Machine Learning techniques, such as the CNN-based Intrusion Detection System (IDS) proposed by Hossain et al. [55], which is

highly focused on achieving impressive detection accuracy (up to 99.99%) but lacks validation in real-world driving scenarios, HyCAN emphasizes a balanced approach by testing all the models on real-world CAN data with various performance metrics.

Moreover, while advanced approaches like CANet [56] and CANintelliIDS [15] have demonstrated strong detection capabilities, they often come with significant computational costs. For example, CANet’s reliance on LSTMs and autoencoders can be computationally expensive and challenging to deploy in resource-constrained environments. Similarly, CANintelliIDS, with its hybrid CNN-GRU architecture, introduces a high level of computational overhead that might limit its practical deployment. In contrast, HyCAN addresses these limitations by incorporating a hybrid framework that selects and fine-tunes models to maintain a high level of performance while minimizing computational demands, making it more suitable for real-time deployment in automotive environments.

Additionally, our research distinguishes itself using real-world CAN data from multiple vehicles. This contrasts with many existing studies that often focus on a single dataset or a specific vehicle, which can limit the applicability of their results. Furthermore, HyCAN goes beyond the conventional approach by evaluating the impact of timestamp features in the detection process, thereby exploring an additional dimension of data that can significantly influence the performance and accuracy of IDS models.

In summary, HyCAN offers a more comprehensive and practical solution for detecting cyberattacks in In-vehicle networks and comparing multiple ML-based approaches at the same time, by integrating a wide range of models and addressing the critical challenges of computational efficiency, real-world applicability across different vehicle types.

CHAPTER 5

Conclusion and Future Work

5.1 Conclusion

This thesis has introduced HyCAN, a Hybrid framework designed to perform comparative performance study to secure In-vehicle networks from various cyber threats and to perform the high performance comparative study by including multiple ML-based IDS techniques in one framework. The study explored the vulnerabilities of modern vehicles, which rely heavily on Electronic Control Units (ECUs), and demonstrated how these systems are susceptible to a range of attacks due to the lack of encryption and authentication within the CAN protocol.

In chapter 1, we began by identifying the increasing risks associated with the rise of sophisticated electronic systems in vehicles. As these systems grow more connected and reliant on ECUs, the potential for cyberattacks, such as DoS and message spoofing, becomes a significant threat. The problem statement focused on the inadequacies of existing IDS and highlighted the need for a robust framework that could detect multiple attack types on real-world vehicular networks. The motivation for this research was derived from the alarming potential of vehicle control systems being compromised, endangering passengers and other road users.

Chapter 2 provided a thorough review of in-vehicle communication systems, with a focus on the CAN protocol and its security weaknesses. We discussed several types of cyber intrusions, such as DoS, spoofing, and fuzzing attacks, to emphasize the current threats vehicles face. The chapter also explored how Machine Learning and Deep Learning techniques are used in intrusion detection, setting the stage for

the hybrid approach developed in this study. Additionally, a literature review was conducted to identify gaps in previous research, helping us shape the objectives of the HyCAN framework.

In chapter 3, the HyCAN framework was implemented by integrating traditional ML models with advanced Deep Learning techniques. The methodology chapter outlined the tools, libraries, dataset, and system configuration used to develop and evaluate the models. Two key research scenarios (RS1 and RS2) were introduced to investigate the relevance of the timestamp feature in detecting attacks. The framework was tested on real-world CAN bus data collected from two different vehicles, the Chevrolet Impala and Subaru Forester, making it one of the first studies to employ such a diverse dataset for intrusion detection. The chapter also detailed the preprocessing steps, including feature selection, data normalization, and balancing, which ensured that the models could handle the complexities of the data.

In chapter 4, the experimental results demonstrated the effectiveness of the HyCAN framework in detecting a variety of attacks across different vehicles. The models were evaluated based on performance metrics such as accuracy, precision, recall, F1-score, and AUC-ROC. It was found that the hybrid approach, particularly the stacked models and transformer-based models, performed exceptionally well in both research scenarios. The results indicated that the timestamp feature did contribute to better detection accuracy, especially for certain attack types. The framework's adaptability was tested by comparing results from two different vehicles, confirming the potential of the proposed IDS to generalize across various vehicular systems. The stacked ensemble methods, which combined the strengths of multiple models, further enhanced the accuracy of the predictions, showing a promising direction for future research in this area.

The research demonstrated that a hybrid machine learning and deep learning-based IDS, such as HyCAN, is not only capable of providing robust and accurate intrusion detection in In-vehicle networks, but also shows how to integrate multiple IDS techniques in one framework. By leveraging both traditional ML models and Deep Neural Networks based architectures, this study addresses the need for a more

comprehensive and scalable solution to vehicular cybersecurity challenges. The inclusion of real-world CAN data in the experiments highlighted the practicality of the system and its ability to operate effectively under different conditions.

5.2 Future Work

While HyCAN has demonstrated promising results in detecting various types of cyberattacks on In-vehicle networks, there are several avenues for future research that could further enhance the capabilities and applicability of this framework.

- **Real-Time Implementation and Testing:** One critical next step is implementing HyCAN in a real-time environment and evaluating its performance under driving conditions. This will help assess the ability of models to detect and respond to attacks in real-world scenarios, where factors such as varying network loads, environmental conditions, and real-time constraints could influence its effectiveness. Real-time testing provides valuable insights into the framework’s latency and impact on vehicle safety and performance.
- **Pre-training and Optimization of Open-Source LLM Models:** As in current research work, BERT showed good performance in detecting attacks, but the only problem is the high computational requirement. So future work will explore pre-training open source LLM models like Meta AI’s LLama on CAN data, by using pruning and quantization techniques to reduce the model size and computational requirements, making these models more suitable for deployment in resource-constrained environments like vehicle ECUs. This approach could lead to more efficient and powerful models that maintain high performance while being lightweight enough for real-time applications in automotive cybersecurity.
- **Enhancing Model Interpretability with xAI:** While HyCAN currently provides strong detection capabilities, improving the interpretability of the underlying models, particularly the deep learning components, would be a valuable

addition. Future work will include techniques such as SHAP or LIME to provide more transparent decision-making processes.

- **Integration with Vehicle-to-Everything (V2X) Communication:** As vehicles become more connected through V2X communication technologies, integrating HyCAN with V2X networks could provide a more comprehensive security solution. Future research could explore how HyCAN can be extended to detect and mitigate threats within the vehicle’s internal network and in its communications with other vehicles, infrastructure, and external networks.
- **Incorporating Advanced Anomaly Detection Techniques:** Future research could explore the integration of more advanced anomaly detection techniques, such as Generative Adversarial Networks or unsupervised learning methods, to enhance the ability of HyCAN to detect previously unseen attack types. This could make the system more resilient against evolving cyber threats that may not have been present in the training data.
- **Adapting to Emerging Protocols:** With the automotive industry gradually transitioning towards newer communication protocols like FlexRay, Automotive Ethernet, and Time-Triggered Ethernet, future work could focus on adapting HyCAN to these protocols. This would involve redesigning and retraining the models to handle these protocols vulnerabilities, ensuring that the IDS remains effective in protecting next-generation vehicles.
- **Expanding the Dataset:** Although the current research has leveraged a diverse dataset collected from multiple vehicles, expanding this dataset to include a broader range of vehicle models, driving conditions, and attack types would further enhance the robustness and generalizability of the IDS. Future work also involve collecting data from more recent vehicle models with different network architectures and communication protocols could help adapt HyCAN to future automotive technologies.

REFERENCES

- [1] Abdulrahman Abu Elkhail et al. “Vehicle security: A survey of security issues and vulnerabilities, malware attacks and defenses”. In: *IEEE Access* 9 (2021), pp. 162401–162437.
- [2] Tong-Jin Park, Chang-Soo Han, and Sang-Ho Lee. “Development of the electronic control unit for the rack-actuating steer-by-wire using the hardware-in-the-loop simulation system”. In: *Mechatronics* 15.8 (2005), pp. 899–918.
- [3] Cesar Bernardini, Muhammad Rizwan Asghar, and Bruno Crispo. “Security and privacy in vehicular communications: Challenges and opportunities”. In: *Vehicular Communications* 10 (2017), pp. 13–28.
- [4] Emad Aliwa et al. “Cyberattacks and countermeasures for in-vehicle networks”. In: *ACM computing surveys (CSUR)* 54.1 (2021), pp. 1–37.
- [5] Rathin Chandra Shit et al. “AI-enabled fingerprinting and crowdsource-based vehicle localization for resilient and safe transportation systems”. In: *IEEE Transactions on Intelligent Transportation Systems* 22.7 (2021), pp. 4660–4669.
- [6] Zhiqiang Cai et al. “0-days & mitigations: Roadways to exploit and secure connected BMW cars”. In: *Black Hat USA* 2019.39 (2019), p. 6.
- [7] Mahdi Dibaei et al. “Attacks and defences on intelligent connected vehicles: A survey”. In: *Digital Communications and Networks* 6.4 (2020), pp. 399–421.
- [8] Hassan N Noura et al. “LoRCA: Lightweight round block and stream cipher algorithms for IoV systems”. In: *Vehicular Communications* 34 (2022), p. 100416.
- [9] Arcangelo Castiglione et al. “Securing the internet of vehicles through lightweight block ciphers”. In: *Pattern Recognition Letters* 135 (2020), pp. 264–270.
- [10] Xuemei Sun et al. “An integrated intrusion detection model of cluster-based wireless sensor network”. In: *PloS one* 10.10 (2015), e0139513.
- [11] Samuel Woo, Hyo Jin Jo, and Dong Hoon Lee. “A practical wireless attack on the connected car and security protocol for in-vehicle CAN”. In: *IEEE Transactions on intelligent transportation systems* 16.2 (2014), pp. 993–1006.

- [12] Jay Nagarajan et al. “Machine Learning based intrusion detection systems for connected autonomous vehicles: A survey”. In: *Peer-to-Peer Networking and Applications* 16.5 (2023), pp. 2153–2185.
- [13] Mohammed Almehdhar et al. “Deep Learning in the Fast Lane: A Survey on Advanced Intrusion Detection Systems for Intelligent Vehicle Networks”. In: *IEEE Open Journal of Vehicular Technology* (2024).
- [14] Md Delwar Hossain et al. “Long short-term memory-based intrusion detection system for in-vehicle controller area network bus”. In: *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE. 2020, pp. 10–17.
- [15] Abdul Rehman Javed et al. “CANintelliIDS: Detecting in-vehicle intrusion attacks on a controller area network using CNN and attention-based GRU”. In: *IEEE transactions on network science and engineering* 8.2 (2021), pp. 1456–1466.
- [16] Prashanth Balaji, Majid Ghaderi, and Hongwen Zhang. “CANLite: Anomaly detection in controller area networks with multitask learning”. In: *2022 IEEE 95th Vehicular Technology Conference: (VTC2022-Spring)*. IEEE. 2022, pp. 1–5.
- [17] Brooke Lampe and Weizhi Meng. “can-train-and-test: A Curated CAN Dataset for Automotive Intrusion Detection”. In: *Computers & Security* 140 (2024), p. 103777.
- [18] H Lenz, CK Wagner, and R Sollacher. “Multi-anticipative car-following model”. In: *The European Physical Journal B-Condensed Matter and Complex Systems* 7 (1999), pp. 331–335.
- [19] Sparsh Sharma and Ajay Kaul. “A survey on Intrusion Detection Systems and Honeypot based proactive security mechanisms in VANETs and VANET Cloud”. In: *Vehicular communications* 12 (2018), pp. 138–164.
- [20] Min-Joo Kang and Je-Won Kang. “Intrusion detection system using deep neural network for in-vehicle network security”. In: *PloS one* 11.6 (2016), e0155781.

- [21] Azeem Hafeez et al. *State of the art survey on comparison of can, flexray, lin protocol and simulation of lin protocol*. Tech. rep. SAE Technical Paper, 2020.
- [22] Saif Al-Sultan et al. “A comprehensive survey on vehicular ad hoc network”. In: *Journal of network and computer applications* 37 (2014), pp. 380–392.
- [23] Fan Yu, Dao-Fei Li, and DA Crolla. “Integrated vehicle dynamics control—state-of-the art review”. In: *2008 IEEE vehicle power and propulsion conference*. IEEE. 2008, pp. 1–6.
- [24] Hyunsung Lee, Seong Hoon Jeong, and Huy Kang Kim. “OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame”. In: *2017 15th Annual Conference on Privacy, Security and Trust (PST)*. IEEE. 2017, pp. 57–5709.
- [25] Omid Avatefipour and Hafiz Malik. “State-of-the-art survey on in-vehicle network communication (CAN-Bus) security and vulnerabilities”. In: *arXiv preprint arXiv:1802.01725* (2018).
- [26] Tazar Hussain et al. “An Attack Impact and Host Importance based Approach to Intrusion Response Action Selection”. In: *Proceedings of the 4th International Conference on Information Technology and Computer Communications*. 2022, pp. 84–91.
- [27] Mehmet Bozdal, Mohammad Samie, and Ian Jennions. “A survey on can bus protocol: Attacks, challenges, and potential solutions”. In: *2018 International Conference on Computing, Electronics & Communications Engineering (iC-CECE)*. IEEE. 2018, pp. 201–205.
- [28] Miroslav Kubat. *An introduction to machine learning*. Springer, 2017.
- [29] Ekaba Bisong and Ekaba Bisong. “Logistic regression”. In: *Building machine learning and deep learning models on google cloud platform: A comprehensive guide for beginners* (2019), pp. 243–250.

- [30] Raphael Couronné, Philipp Probst, and Anne-Laure Boulesteix. “Random forest versus logistic regression: a large-scale benchmark experiment”. In: *BMC bioinformatics* 19 (2018), pp. 1–14.
- [31] Fortunato S De Menezes et al. “Data classification with binary response through the Boosting algorithm and logistic regression”. In: *Expert Systems with Applications* 69 (2017), pp. 62–73.
- [32] Michael Steinbach and Pang-Ning Tan. “kNN: k-nearest neighbors”. In: *The top ten algorithms in data mining*. Chapman and Hall/CRC, 2009, pp. 165–176.
- [33] R Short and Keinosuke Fukunaga. “The optimal distance measure for nearest neighbor classification”. In: *IEEE transactions on Information Theory* 27.5 (1981), pp. 622–627.
- [34] Marcelo Beckmann, Nelson FF Ebecken, and Beatriz SL Pires de Lima. “A KNN undersampling approach for data balancing”. In: *Journal of Intelligent Learning Systems and Applications* 7.04 (2015), pp. 104–116.
- [35] Leo Breiman. “Random forests”. In: *Machine learning* 45 (2001), pp. 5–32.
- [36] Zhongheng Zhang. “Too much covariates in a multivariable model may cause the problem of overfitting”. In: *Journal of thoracic disease* 6.9 (2014).
- [37] Lior Rokach and Oded Maimon. “Decision trees”. In: *Data mining and knowledge discovery handbook* (2005), pp. 165–192.
- [38] Barry De Ville. “Decision trees”. In: *Wiley Interdisciplinary Reviews: Computational Statistics* 5.6 (2013), pp. 448–455.
- [39] Wengang Zhang et al. “Prediction of undrained shear strength using extreme gradient boosting and random forest based on Bayesian optimization”. In: *Geoscience Frontiers* 12.1 (2021), pp. 469–477.
- [40] Tianqi Chen and Carlos Guestrin. “Xgboost: A scalable tree boosting system”. In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016, pp. 785–794.

- [41] Jerome H Friedman. “Greedy function approximation: a gradient boosting machine”. In: *Annals of statistics* (2001), pp. 1189–1232.
- [42] Sam Chapman, Seifey Mohammad, and Kimberly Villegas. “Predicting Listing Prices In Dynamic Short Term Rental Markets Using Machine Learning Models”. In: *arXiv preprint arXiv:2308.06929* (2023).
- [43] Anna Veronika Dorogush, Vasily Ershov, and Andrey Gulin. “CatBoost: gradient boosting with categorical features support”. In: *arXiv preprint arXiv:1810.11363* (2018).
- [44] Liudmila Prokhorenkova et al. “CatBoost: unbiased boosting with categorical features”. In: *Advances in neural information processing systems* 31 (2018).
- [45] Bohdan Pavlyshenko. “Using stacking approaches for machine learning models”. In: *2018 IEEE second international conference on data stream mining & processing (DSMP)*. IEEE. 2018, pp. 255–258.
- [46] Iwan Syarif et al. “Application of bagging, boosting and stacking to intrusion detection”. In: *Machine Learning and Data Mining in Pattern Recognition: 8th International Conference, MLDM 2012, Berlin, Germany, July 13-20, 2012. Proceedings* 8. Springer. 2012, pp. 593–602.
- [47] Weili Jiang et al. “SSEM: A novel self-adaptive stacking ensemble model for classification”. In: *IEEE Access* 7 (2019), pp. 120337–120349.
- [48] Mohammad Humayun Kabir et al. “Network intrusion detection using unsw-nb15 dataset: Stacking machine learning based approach”. In: *2022 International Conference on Advancement in Electrical and Electronic Engineering (ICAEEE)*. IEEE. 2022, pp. 1–6.
- [49] Soundar Ramchandran and R Russell Rhinehart. “Do neural networks offer something for you?” In: *InTech* 42.11 (1995).
- [50] Dennis W Ruck, Steven K Rogers, and Matthew Kabrisky. “Feature selection using a multilayer perceptron”. In: *Journal of neural network computing* 2.2 (1990), pp. 40–48.

- [51] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural computation* 9.8 (1997), pp. 1735–1780.
- [52] Ashish Vaswani. “Attention is all you need”. In: *arXiv preprint arXiv:1706.03762* (2017).
- [53] Sebastian Raschka. *Build A Large Language Model (From Scratch)*. Manning, 2024. ISBN: 978-1633437166. URL: <https://www.manning.com/books/build-a-large-language-model-from-scratch>.
- [54] Achref Haddaji et al. “Investigation of Security Threat Datasets for Intra-and Inter-Vehicular Environments”. In: *Sensors* 24.11 (2024), p. 3431.
- [55] Md Delwar Hossain et al. “An effective in-vehicle CAN bus intrusion detection system using CNN deep learning approach”. In: *GLOBECOM 2020-2020 IEEE global communications conference*. IEEE. 2020, pp. 1–6.
- [56] Markus Hanselmann et al. “CANet: An unsupervised intrusion detection system for high dimensional CAN bus data”. In: *Ieee Access* 8 (2020), pp. 58194–58205.
- [57] Mee Lan Han, Byung Il Kwak, and Huy Kang Kim. “Event-triggered interval-based anomaly detection and attack identification methods for an in-vehicle network”. In: *IEEE Transactions on Information Forensics and Security* 16 (2021), pp. 2941–2956.
- [58] Sooryaa Vignesh Thiruloga, Vipin Kumar Kukkala, and Sudeep Pasricha. “TENET: Temporal CNN with attention for anomaly detection in automotive cyber-physical systems”. In: *2022 27th Asia and South Pacific design automation conference (ASP-DAC)*. IEEE. 2022, pp. 326–331.
- [59] Minki Nam, Seungyoung Park, and Duk Soo Kim. “Intrusion detection method using bi-directional GPT for in-vehicle controller area networks”. In: *IEEE Access* 9 (2021), pp. 124931–124944.
- [60] Mahdi Bahaghighat, Leila Akbari, and Qin Xin. “A machine learning-based approach for counting blister cards within drug packages”. In: *IEEE Access* 7 (2019), pp. 83785–83796.

- [61] Michael Buckland and Fredric Gey. “The relationship between recall and precision”. In: *Journal of the American society for information science* 45.1 (1994), pp. 12–19.
- [62] Zachary Chase Lipton, Charles Elkan, and Balakrishnan Narayanaswamy. “Thresholding classifiers to maximize F1 score”. In: *arXiv e-prints* (2014), arXiv–1402.
- [63] Sarang Narkhede. “Understanding auc-roc curve”. In: *Towards data science* 26.1 (2018), pp. 220–227.

VITA AUCTORIS

NAME:	Vikrant
PLACE OF BIRTH:	Kurukshetra, Haryana, India
EDUCATION:	B.Tech in Computer Science & Engineering, University Institute of Engineering & Technology , Kurukshetra University, Kurukshetra, Haryana, India, 2019 M.Sc. Computer Science, University of Windsor, Ontario, Canada, 2024